

深度学习导论

基于第四章「神经网络基础（一）」101页 + 第五章「神经网络基础（二）」95页

第一部分：神经网络基础（第四章）

一、多层感知机（MLP）基本结构

1. 从感知机到多层感知机

感知机回顾（第4-8页）：

1943年 McCulloch-Pitts 模型——第一个神经元数学模型。固定权重，无学习能力。1957年 Rosenblatt 发明感知机，可根据错误反馈学习。感知机对每个输入加权求和，与阈值比较——超过阈值输出1，否则输出0。

感知机的数学形式：

- 输入 $x_1, x_2, \dots, x_d$ ，权重 $w_1, w_2, \dots, w_d$ ，偏置 b
- 输出 = $\text{sign}(w_1x_1 + \dots + w_dx_d + b)$

感知机算法：

1. 初始化权重 $w = 0, b = 0$
2. 遍历训练集，找到误分类样本（即 $y \cdot (w^T x + b) \leq 0$ 的样本）
3. 更新参数： $w = w + y \cdot x, b = b + y$
4. 对于线性可分数据，该算法从理论上被证明会收敛

线性模型的局限——无法解决异或问题（第12页）：

感知机是一条直线（或超平面），无法分割异或（XOR）问题。Marvin Minsky 和 Seymour Papert 在 1969 年的著作《Perceptrons》中指出了这一致命缺陷——异或问题无法用单层感知机解决，因为找不到一条直线把两类分开。

2. MLP 的结构（第13-16页）

定义：多层感知机是由输入层、一个或多个隐藏层、输出层组成的前馈神经网络。

- 输入层：每个神经元对应数据的一个特征，无需计算
- 隐藏层 & 输出层：假设当前为第 ℓ 层，有 d_ℓ 个神经元

每个神经元的前向计算公式：

$$z_j^{(\ell)} = \sum_{i=1}^{d_{\ell-1}} w_{ji}^{(\ell)} x_i^{(\ell-1)} + b_j^{(\ell)}$$
$$a_j^{(\ell)} = \sigma(z_j^{(\ell)})$$

其中：

- $w_{ji}^{(\ell)}$ = 第 ℓ 层第 j 个神经元赋予前一层第 i 个神经元的权重
- $b_j^{(\ell)}$ = 第 ℓ 层第 j 个神经元的偏置
- $\sigma(\cdot)$ = 激活函数

- $d_{\ell-1}$ = 前一层的神经元个数

矩阵形式（第22页）：

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}$$

$$a^{(\ell)} = \sigma(z^{(\ell)})$$

其中 $W^{(\ell)}$ 是 $d_{\ell} \times d_{\ell-1}$ 的权重矩阵， $b^{(\ell)}$ 是 d_{ℓ} 维的偏置向量。

关键洞察（第26页）：神经网络的每一次前向操作本质上就是**矩阵乘法 + 激活函数**。GPT-3（1750亿参数）需要约1400块 A100 GPU 训练一个月——矩阵乘法的并行计算能力使得这一切成为可能。

3. 通用近似定理（第28-33页）

定理内容（Cybenko, 1989）：当激活函数是 sigmoid 型函数时，**只有单隐层的多层感知机就可以在任意精度下逼近定义在紧集上的任意连续函数。**

理论意义：

- 证明了神经网络在理论上具有表示任意复杂函数的能力——回答了「神经网络能不能」这个根本问题
- 非线性激活函数是关键：如果没有激活函数，多层感知机等价于线性模型（无论多少层，最终都是 $Wx+b$ ）

深度 vs 宽度（第31-33页）：

Telgarsky (2016) 和 Eldan & Shamir (2016) 在 COLT 上发表的结果：

- 存在一类函数，用深度为 3 层的网络可以用多项式级参数精确表示
- 但如果限制深度为 2 层（浅层），所需神经元数必须是指数级

两者的对比：

- Cybenko 回答的是存在性（「能不能」）——对**所有**连续函数，足够宽的单隐层网络可以任意精度逼近
- Telgarsky/Eldan-Shamir 回答的是效率性（「省不省」）——对**某些特定**函数，深度可以**指数级**节省参数

实验验证（第34页）：ImageNet 上从 AlexNet(8层, 16.4%) → VGG(19层, 7.3%) → GoogLeNet(22层, 6.7%) → ResNet(152层, 3.57%)，深度增加持续带来性能提升。

二、激活函数（第17-20页）

为什么需要激活函数

单层感知机使用阈值函数 $\text{sign}(z)$ ，问题：1. 函数不连续导致难以优化；2. 二值化限制模型能力（假设空间变小）。

Sigmoid 函数

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

- 输出范围 (0, 1)，可解释为概率
- 连续光滑，适合梯度优化
- **缺点：**输入很大或很小时，梯度趋近于 0（饱和区），容易导致**梯度消失**
- 导数： $\sigma'(x) = \sigma(x)(1-\sigma(x)) \in (0, 0.25]$

Tanh 函数

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- 输出范围 $(-1, 1)$ ，零中心化
- 同样存在饱和区，但比 Sigmoid 稍好

ReLU 函数（实践中非常有效！）

$$\text{ReLU}(x) = \max(0, x)$$

- 正半轴导数恒为 1，**不会出现梯度消失**
- 计算极其简单（只有比较操作）
- 非负半轴恒为 0，导致一部分神经元可能「死亡」（Dying ReLU）
- 连续但不可导（ $x=0$ 处），实践中不影响训练

Leaky ReLU 函数

$$\text{LeakyReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{otherwise} \end{cases}$$

- α 一般较小（如 0.1），参数亦可学习（PReLU）
- 解决了 ReLU 的「死亡」问题——负半轴仍保留微小梯度

仿真实验验证（第94-96页）

隐藏层 6 神经元、4 层时：ReLU 和 Tanh 学习效果良好；Sigmoid 几乎没有学习——因为深层网络中的梯度消失使 Sigmoid 完全无法有效训练。

三、损失函数（第36-49页）

损失函数的作用

损失函数本质上是一个可量化的评价准则——将模型的预测结果与真实目标之间的差距压缩成一个标量值：

1. 定义了「好」的数学内涵（不同任务对「好」的定义不同）
2. 提供了优化的方向（梯度指明了每个参数应当调整的方向）
3. 隐式嵌入了先验知识（交叉熵对应最大似然估计，MSE 假设高斯噪声）

多变量回归任务的损失函数（第39页）

给定数据集 $\{(x_i, y_i)\}$ ，其中 $x_i \in \mathbb{R}^d, y_i \in \mathbb{R}^p$ 。MLP 输出 $\hat{y} = \text{MLP}(x)$ 。**均方误差损失：**

$$\mathcal{L}(y, \hat{y}) = \frac{1}{n} \sum_{i=1}^n \|y_i - \hat{y}_i\|^2$$

或 L1 损失（绝对误差）： $\mathcal{L}(y, \hat{y}) = \frac{1}{n} \sum_{i=1}^n \|y_i - \hat{y}_i\|_1$

多分类任务的损失函数——交叉熵（第40-48页）

对于多分类任务， $y \in \{1, 2, \dots, C\}$ 。

Softmax 函数：将 MLP 输出的 logits 映射为概率分布

$$P(y = c|x) = \frac{\exp(z_c)}{\sum_{j=1}^C \exp(z_j)}$$

One-hot 编码：将类别标签 y 转换为 C 维向量 $[0, \dots, 1, \dots, 0]$ ，只有正确类别位置为 1。

交叉熵损失 Cross-Entropy Loss：

$$\mathcal{L}(y, \hat{y}) = - \sum_{c=1}^C y_c \log \hat{y}_c$$

其中 $y_c \in \{0, 1\}$ 是真实标签（One-hot）， $\hat{y}_c$ 是 Softmax 输出的预测概率。

等价于最大化正确类别的对数似然——让正确类别的预测概率尽可能大。

四、梯度计算（第51-82页）

经验风险最小化（第51页）

训练 MLP 的目标是**经验风险最小化**（以多分类为例）：

$$\min_{W,b} \mathcal{L}(W,b) = -\frac{1}{n} \sum_{i=1}^n \sum_{c=1}^C y_{i,c} \log \hat{y}_{i,c}$$

其中 $\hat{y}$ 由 MLP 的权重 W 和偏置 b 以及输入 x 确定。

梯度下降法（第53-56页）

1. 随机初始化参数
2. 计算损失 $\nabla L(w)$ （梯度）
3. 更新参数： $w \leftarrow w - \eta \nabla L(w)$
4. 重复 2-3 直到梯度接近零

梯度 $\nabla f(x)$ 的几何意义：

- **方向：**函数 f 在点 x 处**上升最快**的方向。负梯度方向 $(-\nabla f(x))$ 是函数值**下降最快**的方向
- **大小：** $\|\nabla f(x)\|$ 表示在该方向上的变化率——模越大越「陡峭」、越小越「平坦」

链式法则（第67页）

复合函数 $h(x) = f(g(x))$ 的导数： $h'(x) = f'(g(x)) \cdot g'(x)$

链式法则是神经网络基于梯度反向传播进行训练的核心——损失函数对参数的梯度可以分解为从输出到输入的多个局部梯度的乘积，每个局部梯度容易计算。

多层感知机的梯度计算——反向传播（第70-82页）

前向传播：对于每个数据，输入 $x^0 \rightarrow$ 逐层计算 $z^{(\ell)}$ 和 $a^{(\ell)} \rightarrow$ 输出 $\rightarrow$ 损失 ℓ

反向传播：自顶向下地计算：

1. 计算损失对输出的梯度 $\partial \ell / \partial a^{(L)}$

2. 计算每一层的局部导数 $\partial a^{(\ell)}/\partial z^{(\ell)} = \text{diag}(\sigma'(z^{(\ell)}))$ (激活函数的导数)

3. **梯度计算**: $\frac{\partial \ell}{\partial W^{(\ell)}} = \frac{\partial \ell}{\partial z^{(\ell)}} \cdot (a^{(\ell-1)})^T$

4. **误差传播**: $\frac{\partial \ell}{\partial z^{(\ell-1)}} = (W^{(\ell)})^T \cdot \frac{\partial \ell}{\partial z^{(\ell)}} \odot \sigma'(z^{(\ell-1)})$

梯度消失/爆炸分析 (第78-82页):

$$\frac{\partial \ell}{\partial W^{(\ell)}} \propto \prod_{k=\ell+1}^L (W^{(k)})^T \cdot \text{diag}(\sigma'(z^{(k)}))$$

- 若 $\|W^{(k)} \cdot \sigma'(z^{(k)})\| < 1$, 连乘 $\rightarrow 0$: **梯度消失** (Sigmoid/Tanh 容易发生)
- 若 $\|W^{(k)} \cdot \sigma'(z^{(k)})\| > 1$, 连乘 $\rightarrow \infty$: **梯度爆炸** (权重初始化过大)

不同激活函数的梯度特性 (第79-81页):

- **Sigmoid**: $\sigma'(x) = \sigma(x)(1-\sigma(x)) \in (0, 0.25]$, 极易梯度消失
- **Tanh**: 类似 Sigmoid, 导数也在 0 附近
- **ReLU**: $\sigma'(x) = 1$ ($x > 0$ 时), 梯度消失风险大幅降低; 但 $x \leq 0$ 时梯度为 0
- **Leaky ReLU**: $x > 0$ 时导数为 1, $x \leq 0$ 时为 α (小正数) ——几乎不会完全消失

五、优化基础 (第83-93页)

凸函数与凸优化 (第83-89页)

凸函数的定义 (两种等价形式):

1. 函数图像上的任意两点连线在函数图像之上: $f(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda)f(y)$, $\forall \lambda \in [0,1]$
2. 可微时: $f(y) \geq f(x) + \nabla f(x)^T(y-x)$, $\forall x, y \in \text{dom } f$

凸性: 解决了「往哪里走」的问题——负梯度方向是下降方向, 且最终能收敛到**全局最优**。

平滑性: 解决了「走多快」的问题——限制了梯度的变化幅度, 使我们能安全地选择步长并量化收敛速度。

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|, \forall x, y \in \text{dom } f$$

凸优化的收敛: 梯度下降在凸且平滑的函数上, 经过 T 次迭代后, $f(\bar{w}) - f(w^*) = O(1/T)$

为什么深度学习不用梯度下降 (第90-93页)

两个原因:

1. **非凸性**: 神经网络的损失函数不再具备凸性, 理论保证弱于凸情况 (只保证能收敛到梯度很小的点, 不保证全局最优)
2. **计算开销过大**: ImageNet 有 1.4×10^7 张图, 每次梯度下降需要遍历全部数据计算梯度

随机梯度下降 (SGD) 解决计算开销问题:

$$w_{t+1} = w_t - \eta \cdot \tilde{\nabla} \mathcal{L}(w_t)$$

其中 $\tilde{\nabla} \mathcal{L}$ 是无偏估计——从数据集中均匀采样一个大小为 B 的子集 (Batch) 来计算梯度。 $B \ll n$, 每次迭代的时间复杂度从 $O(n)$ 降至 $O(B)$ 。

- 凸函数下 SGD 收敛速度为 $O(1/\sqrt{T})$
- 非凸平滑函数下 SGD 保证最终梯度的模 $\rightarrow 0$
- **实践经验**: SGD 的随机性**有助于跳出局部最优**——这是梯度下降不具备的优势

第二部分：神经网络基础（第五章）

六、数据准备

1. 归一化 Normalization（第8-10页）

定义：将不同量纲或不同取值范围的特征变换到可比较的尺度上，避免大数值特征主导训练，提高训练稳定性和公平性。

为什么需要归一化：未归一化时，大小特征混合会导致梯度下降出现「锯齿状」路径（Zigzag）——某些维度梯度过大、某些过小，收敛极慢。

标准化 Standardization（最常用）：

$$\hat{x}_j = \frac{x_j - \mu_j}{\sigma_j}$$

其中 μ_j 是第 j 个特征的均值， σ_j 是标准差。

Min-Max 归一化（对异常值敏感）：

$$\hat{x}_j = \frac{x_j - x_j^{\min}}{x_j^{\max} - x_j^{\min}}$$

将特征映射到 [0, 1]。

2. 数据增强 Data Augmentation（第11-15页）

定义：通过对原始样本进行旋转、翻转、裁剪、加噪声等变换，构造更多训练样本，旨在扩大数据分布、提升模型泛化能力。

图像数据增强方法：

方法	描述
翻转	水平翻转、垂直翻转或同时翻转
旋转	以某一个角度旋转图像
缩放	增大或减小图像尺寸
裁剪	裁剪图像的一个子区域
平移	将图像水平/垂直移动
噪声注入	在图像中加入噪声
颜色空间	改变图像颜色通道（如调整亮度、对比度、饱和度）
锐化	修改图像清晰度

图像擦除：从训练图像中抹去一部分信息——迫使模型关注其它相关部分，而不仅仅是最具判别力的部分（防止模型「只看猫脸不看猫身」）。

图像混合：

- CutMix：将一张图的一部分贴到另一张图上
- MixUp：按比例像素融合两张图
- 通过频率空间操作生成形状光滑自然的掩码

文本数据增强：

- 回译法：原文 → 翻译成另一种语言 → 翻译回原语言（产生同义但措辞不同的变体）
- 随机词替换/插入/交换/删除：对原文加入微小扰动构造变体

3. 类型转换（第16-26页）

核心原则：神经网络模型通常要求输入数据的特征为**连续数值型**。

数值型数据 Numerical Data：连续有序，如图像的像素值（0-255），无需额外转换。

类别型数据 Categorical Data：离散无序，需要编码为数值：

- 二分类：0/1 或 -1/1
- 多分类：**独热编码 One-Hot Encoding**——C 维指示向量，只有正确类别位置为 1，其余全 0

文本数据 Text Data 的处理流程：

1. **分词 Tokenization**：将文本切分成最小语义单位 Token
2. **移除停止词**：删除「的」「了」「the」「a」等高频但无语义的词
3. **提取词干**：「running」「runs」「ran」→「run」
4. **词袋模型 BoW**：记录文本中各词典词的频率 → 丢失语序（「I love you」和「You love me」的 BoW 向量完全相同）
5. **嵌入 Embedding**：将每个词映射为稠密向量再整合成矩阵
6. **处理变长输入**：Padding（短句补零）、Truncation（长句截断）、Pooling（压缩为固定长度向量）、RNN/Transformer（原生支持变长序列，通过 Masking 忽略填充位）

BPE 字节对编码（当前最主流的子词分词法）：不断合并出现频率最高的相邻字符对，直到达到预设词表大小。解决 OOV（未知词）问题，词表大小可控。

第三部分：模型构建（第五章）

七、丢弃层 Dropout（第28-33页）

核心思想

Dropout 是一种缓解过拟合的关键技术——训练过程中**随机丢弃神经网络中的一些神经元及其连接**，防止神经元之间产生过强的**共适应 Co-adaptation**。

「共适应」指的是一些神经元总是协同工作，产生依赖性（一个神经元的权重是为了弥补另一个神经元的错误），导致模型过于依赖训练数据的特定组合，产生严重过拟合。

数学表达

设第 ℓ 层使用 Dropout:

训练时: $\tilde{a}^{(\ell)} = r^{(\ell)} \odot a^{(\ell)}$

其中 $r^{(\ell)} \sim \text{Bernoulli}(p)$ ——每个神经元以概率 p 保留、以概率 $1-p$ 被「丢弃」(输出置零)。

测试时: **不做随机丢弃**, 而是对所有激活值进行缩放: $a_{test}^{(\ell)} = p \cdot a^{(\ell)}$

测试时的缩放使输出范围和训练时对齐——保证训练和测试的期望输出一致。

Dropout 的三大作用

1. **降低模型容量**: 每次训练都相当于在一个更小的子网络上进行, 减少了有效参数量
2. **破坏关联性**: 强迫神经元独立工作——每次训练随机丢弃不同神经元, 任何两个神经元不能形成固定依赖
3. **模型集成效应**: 每次随机丢弃生成了不同的子网络, 训练时训练了数万个「子网络」, 测试时相当于对这些子网络平均——近似集成学习

八、批归一化 Batch Normalization (第34-49页)

动机——内部协变量偏移

内部协变量偏移 Internal Covariate Shift: 训练过程中, 前层参数改变导致后层神经元输入分布不断变化。网络层数增加会加剧这种影响——后层需要不断适应新的输入分布, 导致训练变慢、不稳定, 容易出现梯度爆炸/消失。

批归一化的做法

基于小批量 Mini-batch 数据:

1. 计算小批量均值 μ_B 和方差 σ_B^2 :

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i, \quad \sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$$

2. 标准化: $\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$

3. **可学习的缩放和平移**: $y_i = \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i)$

γ 和 β 是可学习参数—— γ 拉伸或压缩 (恢复表达能力), β 控制整体平移。

如果只有标准化 (没有 γ, β): 各层的输出被严格限制在均值为 0、方差为 1 的分布中, **削弱了非线性激活函数的表达能力**。 γ 和 β 让网络自己找到最优的分布。

关于 ICS 的最新认识 (第39-41页)

BN 并非主要通过解决 ICS 来提升性能——向 BN 网络中加入随机噪声使 ICS 更严重的实验中, 加了噪声的 BN **性能仍然优于未使用 BN 的网络**。

BN 的真正作用: 使损失函数更平滑 (Lipschitz 连续)——损失函数的梯度和梯度的梯度都被控制在一个常量范围内。这使得可以用**更大的学习率**训练, 从而加快收敛。

测试阶段的 BN（第48-49页）

测试时使用训练阶段累积的全局均值和方差（对整个训练过程中的所有批次求滑动平均）：

$$\mathbb{E}[x] = \mathbb{E}[\mu_B], \quad \text{Var}[x] = \frac{m}{m-1} \mathbb{E}[\sigma_B^2]$$

测试阶段的 BN 输出：

$$y = \frac{\gamma}{\sqrt{\text{Var}[x] + \epsilon}} \cdot x + \left(\beta - \frac{\gamma \cdot \mathbb{E}[x]}{\sqrt{\text{Var}[x] + \epsilon}} \right)$$

九、层归一化 Layer Normalization 与其他归一化（第50-54页）

层归一化 Layer Normalization

对某一层不同神经元的输入进行归一化（沿特征维度）：

$$\mu = \frac{1}{H} \sum_{i=1}^H x_i, \quad \sigma^2 = \frac{1}{H} \sum_{i=1}^H (x_i - \mu)^2$$

$$y = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta \equiv \text{LN}_{\gamma, \beta}(x)$$

与 BN 的区别：BN 沿 batch 维度（不同样本的同一特征），LN 沿 feature 维度（同一样本的不同特征）。LN 不依赖 batch size——适用于任意大小的 batch 甚至 batch size=1，特别适合 RNN/Transformer。

四种归一化对比

方法	归一化维度	适用场景
Batch Norm	沿 Batch × H × W（跨样本）	CNN，需要较大 batch size
Layer Norm	沿 Channel × H × W（跨特征）	RNN / Transformer，不依赖 batch size
Instance Norm	沿 H × W（单样本单通道）	图像风格迁移——消除内容图原风格
Group Norm	通道分组后，组内沿 H × W	batch size 极小时的 CNN 替代方案

十、参数初始化（第55-66页）

重要性

好的参数初始化对优化算法的收敛起到**关键作用**——神经网络面临非凸目标函数，不同初始化可能收敛到完全不同的局部最优解。

为什么不能全 0 初始化

「对称权重」问题：同一层所有神经元参数相同 → 前向传播和反向梯度都完全相同 → 等价于该层只有一个神经元，丧失了多层多神经元的表达能力。

小随机数初始化的问题——方差传播

对于无偏置项的线性层 $a^{(\ell)} = W^{(\ell)} a^{(\ell-1)}$ ，假设输入和权重满足独立同分布（均值为 0）：

$$\text{Var}[a^{(\ell)}] = d_{\ell-1} \cdot \text{Var}[W^{(\ell)}] \cdot \text{Var}[a^{(\ell-1)}]$$

如果 $d_{\ell-1} \cdot \text{Var}[W^{(\ell)}] < 1 \rightarrow$ 方差逐层衰减 $\rightarrow$ 激活值趋向 0 (梯度消失)。

如果 $d_{\ell-1} \cdot \text{Var}[W^{(\ell)}] > 1 \rightarrow$ 方差逐层放大 $\rightarrow$ 激活值爆炸 (梯度爆炸)。

Xavier 初始化 Glorot Initialization (第64页)

目标：控制前向传播和反向传播的方差都不衰减。

假设条件： $\tanh$ 等近似线性的对称激活函数。

前向方差控制要求： $d_{\ell-1} \cdot \text{Var}[W^{(\ell)}] \approx 1 \rightarrow \text{Var}[W^{(\ell)}] = 1 / d_{\ell-1}$

反向方差控制要求： $d_{\ell} \cdot \text{Var}[W^{(\ell)}] \approx 1 \rightarrow \text{Var}[W^{(\ell)}] = 1 / d_{\ell}$

Xavier 初始化的折中方案： $\text{Var}[W^{(\ell)}] = \frac{2}{d_{\ell} + d_{\ell-1}}$

可从均匀分布 $U[-\sqrt{6/(d_{\ell} + d_{\ell-1})}, \sqrt{6/(d_{\ell} + d_{\ell-1})}]$ 采样，或从 $N(0, 2/(d_{\ell} + d_{\ell-1}))$ 采样。

He 初始化 Kaiming Initialization (第65页)

目标：针对 **ReLU** 这种非对称激活函数 (正半轴保留、负半轴清零，方差只传递一半)。

前向方差控制： $\text{Var}[W^{(\ell)}] = \frac{2}{d_{\ell-1}}$

从均匀分布 $U[-\sqrt{6/d_{\ell-1}}, \sqrt{6/d_{\ell-1}}]$ 采样，或从 $N(0, 2/d_{\ell-1})$ 采样。

效果对比 (第66页)：小随机数初始化 ($\sigma=0.01$) 让 50 层网络的激活值全部坍缩到 0；He 初始化让各层激活值保持合理分布——即使层数增加也远大于 0。

第四部分：训练与优化 (第五章)

十一、优化器 (第68-86页)

SGD 的三大问题 (第72-73页)

- 参数无区分：**对所有参数用同一个全局学习率——深层网络中不同层的梯度大小可能差几个数量级，统一学习率导致某些层更新过快或过慢
- 稀疏特征处理差：**低频特征的参数更新次数少，全局学习率难以适配
- 随机梯度方差大：**mini-batch 梯度是真实梯度的有噪估计，方差导致更新方向不稳定、损失曲线震荡严重

AdaGrad——自适应学习率 (第74-79页)

核心思想：给不同参数分配自适应的学习率——利用历史梯度大小动态调整每个参数的学习率。

$$G_t = G_{t-1} + g_t \odot g_t \quad (\text{累积梯度平方})$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \odot g_t$$

- 频繁更新的参数 $\rightarrow G_t$ 大 $\rightarrow$ 学习率 $\eta/\sqrt{G_t}$ 小
- 稀疏更新的参数 $\rightarrow G_t$ 小 $\rightarrow$ 学习率相对大

优点：自动适应参数频率，对稀疏数据友好。

致命缺陷： G_t 是单调递增的 (始终累加正数)，学习率持续单调下降最终趋近于 0，导致后期无法继续学习。

RMSProp——解决 AdaGrad 的衰减问题（第80页）

引入指数滑动平均替代累加，避免学习率单调递减：

$$G_t = \rho G_{t-1} + (1 - \rho) g_t \odot g_t$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \odot g_t$$

ρ 通常取 0.9，旧的梯度平方信息指数衰减——不再单调递减。

Momentum——动量法（第77页）

核心思想：保留一部分之前的更新方向，就像物理中的动量（惯性）——球滚下山时不断加速。

$$v_t = \rho v_{t-1} + (1 - \rho) g_t \quad (\text{动量})$$

$$w_{t+1} = w_t - \eta v_t$$

- 在梯度方向一致的维度上加速
- 在梯度方向震荡的维度上抵消（降低震荡）

Adam——一阶动量 + 二阶自适应 + 去偏（第82-84页）

★ 当前使用最广泛、最推荐的优化器。

三个核心组件：

1. **一阶矩（动量）** $m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ —— 保留历史梯度方向（动量）
2. **二阶矩** $v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ —— 自适应调整学习率（类似 RMSProp）
3. **去偏校正：**由于初始 $m_0 = 0, v_0 = 0$ ，早期估计偏向 0。除以 $(1 - \beta^t)$ 校正

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \odot \hat{m}_t$$

推荐超参数（原文建议）： $\eta=0.001, \beta_1=0.9, \beta_2=0.999, \epsilon=10^{-8}$ 。

Adam 的理论分析问题（第85-86页）：原论文的收敛证明存在错误（二阶矩不一定单调递增）。AMSGrad 修复了这个问题：用历史最大二阶矩替代当前二阶矩（ $\hat{v}_t = \max(\hat{v}_{t-1}, v_t)$ ），保证有效学习率单调递减。

十二、权重衰减 Weight Decay（第87-88页）

原理

权重衰减在每次参数更新时额外将权重乘以一个略小于 1 的因子 $(1 - \eta\lambda)$ ：

$$w_{t+1} = (1 - \eta\lambda)w_t + \Delta w_t$$

对于 SGD（ $\Delta w = -\eta g$ ）：

$$w_{t+1} = (1 - \eta\lambda)w_t - \eta g_t$$

当 λ 为常数时，权重衰减等价于 **L2 正则化**（对损失函数加二范数惩罚）：

$$\mathcal{L}_{Reg}(w) = \mathcal{L}(w) + \frac{\lambda}{2} \|w\|^2$$

AdamW——Adam + 解耦的权重衰减（第88页）

标准 Adam 中，权重衰减和自适应学习率耦合在一起（因为梯度 g 被 v_t 归一化后再减权重衰减）——不再等价于 L2 正则化。

AdamW 的解耦：将权重衰减从自适应梯度更新中分离出来：

$$w_{t+1} = (1 - \eta\lambda)w_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

实践中 AdamW 显著优于标准 Adam 和 SGD + Momentum。

十三、学习率技巧（第89-94页）

学习率衰减（第89页）

优化早期用较大学习率加速收敛，随后逐渐调低学习率以精细收敛到最优解：

- **分段衰减：**每经过一定迭代次数 T_k ，学习率乘以衰减因子
- **逆时衰减：** $\eta_t = \frac{\eta_0}{1+\gamma t}$
- **指数衰减：** $\eta_t = \eta_0 \cdot \gamma^t$
- **自然指数衰减：** $\eta_t = \eta_0 \cdot e^{-\gamma t}$
- **余弦衰减（当前大模型训练主流）：** $\eta_t = \frac{\eta_0}{2} \left(1 + \cos\left(\frac{t\pi}{T}\right)\right)$

Warmup 预热（第90页）

设定预热步数 T_{warmup} ，前 T_{warmup} 步学习率从 0 线性（或指数）增长到初始学习率：

$$\eta_t = \eta_0 \cdot \frac{t}{T_{warmup}}, \quad t = 1, 2, \dots, T_{warmup}$$

为什么需要 Warmup：训练初期模型参数是随机初始化的，梯度方向不稳定。如果一开始就用大学习率，可能导致模型被「炸飞」到不合理的参数空间。Warmup 给模型一个「适应期」——先以小步探索、逐渐加速。

周期性学习率 CLR（第91-94页）

动机：神经网络训练可能陷入局部最优或鞍点区域。周期性地增大学习率有助于跳出这些区域——就像摇晃一下，帮助逃离局部陷阱。

做法：让学习率在 $[\eta_{min}, \eta_{max}]$ 之间周期性变化（如三角波）。这近似于 Snapshot Ensemble 的思想——训练一个模型的同时免费获得多个模型的集成效果（不同周期的不同局部最优的 snapshot 集成）。

第五部分：机器学习基础（第二章 & 第三章）

来源：第二章「机器学习基础（一）」78页 + 第三章「机器学习基础（二）」65页，主讲：毛玉仁

十五、机器学习的定义与基本概念（第二章 第1-27页）

1. 机器学习的定义（第6页）

Tom Mitchell (CMU) 的经典定义：

"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E."

翻译：一个计算机程序，如果它在任务 T 上的表现（用 P 衡量），随着经验 E 的增加而提高，则称它从经验 E 中学习了。

三个核心要素：任务 T、经验 E、性能度量 P。

2. 机器学习的五个要素（第11页）

机器学习包含五个要素——可类比人类学习：

人类学习	机器学习	含义
经验知识	训练数据	用于训练的样本集合（数量和质量都影响性能）
学习载体	假设类 H	所有可能模型的集合（如神经网络参数空间）
常识	归纳偏置	限制对某些假设的选择（如 CNN 的局部感受野）
学习方法	学习算法	优化损失的方法（梯度下降等）
学习目标	学习范式	监督/无监督/强化学习等

3. 机器学习的完整过程（第12页）

在某种学习范式下，基于训练数据（经验），利用学习算法，从受归纳偏置限制的假设类中选取可以达到学习目标的假设，该假设可以泛化到未知数据上。

4. 训练数据（第14页）

训练数据 S 的数量和质量都会影响机器学习性能：

- 数量：在一定范围内显著影响性能
- 质量：多样性、噪声等因素

5. 假设类（第15页）

假设类 H 是所有可能机器学习模型的集合（候选模型的参数空间）。当前大语言模型主要选用神经网络作为假设空间。

6. 归纳偏置（第16页）

归纳偏置限制对某些假设进行选择。例如：

- SVM：最大化训练样本到超平面的距离
- CNN：局部感受野（局部连接）

- RNN：时间序列依赖
- Transformer：稀疏注意力

7. 学习范式（第17页）

三大范式：

- **监督学习**：给定样本及其标签，完成分类/回归
- **无监督学习**：无标签，自组织学习
- **强化学习**：从探索中学习（试错+奖励）

8. 学习目标与经验风险最小化（第18页）

经验风险最小化 ERM 是最常见的学习目标——旨在最小化模型在训练集上的错误。不同于 ERM，K-Means 等无监督方法使用**相对距离最小化**作为目标。

9. 损失函数（第19-20页）

损失函数衡量模型在对应样本上的错误程度。大多数损失函数是真实错误程度的**代理损失 Surrogate Loss**（因为 0-1 损失不可导）。损失函数在训练集上结果的加权和称为**训练损失**。

10. 泛化误差（第22-23页）

机器学习的真正目的在于减小**泛化误差**（Generalization Error），即真实误差。实践中用**验证集**估计泛化误差（超参选择），用**测试集**检验泛化误差（最终评测）。

十六、线性回归（第三章 第10-11页）

监督学习中的回归任务——给定数据集 $D = \{(x_i, y_i)\}$ ，其中 $x_i \in \mathbb{R}^d$ ， $y_i \in \mathbb{R}$ 。目标是学习从特征 x 到连续标签 y 的映射。

线性回归模型： $f(x) = w^T x + b$ （一个最简单的一维线性假设）

十七、可学习性——PAC 理论（第二章 第28-41页）

1. PAC 理论框架

PAC（Probably Approximately Correct，概率近似正确）是刻画机器学习能力的抽象框架。

基本设定：

- 数据集 $D = \{(x_i, y_i)\}$ ， $x_i \in \mathcal{X}$ ， $y_i \in \mathcal{Y} = \{-1, +1\}$ （二分类）
- **独立同分布假设**： $\mathcal{X}$ 服从隐含未知分布 $\mathcal{D}$ ，所有样本独立从 $\mathcal{D}$ 采样（ $D \sim \mathcal{D}^m$ ）

泛化误差： $E(h) = P_{x \sim \mathcal{D}}[h(x) \neq y]$ （学习的终极目标是最小化泛化误差）

经验误差： $\hat{E}(h) = (1/m) \sum \mathbb{I}[h(x_i) \neq y_i]$ （在训练集上可计算的错误率）

重要性质：对于给定映射 h ，经验误差的期望等于泛化误差（ $\mathbb{E}_D[\hat{E}(h)] = E(h)$ ）——经验误差是泛化误差的**无偏估计**。

2. 核心概念（第30-31页）

- **概念 Concept**: 从特征空间 $\mathcal{X}$ 到标记空间 $\mathcal{Y}$ 的映射 c 。目标概念 c : 对所有数据都有 $c(x) = y$
- **概念类 $\mathcal{C}$** : 所有概念构成的集合
- **假设空间 $\mathcal{H}$** : 学习算法考虑的所有可能概念的集合
- **假设 h** : $h \in \mathcal{H}$ 不一定等于目标概念
- **可分性**: 若 $c^* \in \mathcal{H} \rightarrow$ 可分的; $c^* \notin \mathcal{H} \rightarrow$ 不可分的

3. PAC 辨识（第33页）

给定误差参数 ϵ 和置信参数 δ 。对任意目标概念 $c \in \mathcal{C}$ 和分布 $\mathcal{D}$ ，如果存在学习算法 $\mathcal{L}$ 输出 $h \in \mathcal{H}$ 满足：

$$P[E(h) \leq \epsilon] \geq 1 - \delta$$

则称 $\mathcal{L}$ 能从 $\mathcal{H}$ 中 **PAC 辨识** 概念类 $\mathcal{C}$ 。即：**以大概率 ($\geq 1-\delta$) 学得误差不超过 ϵ 的模型。**

4. PAC 可学习（第34页）

如果存在学习算法和多项式 $\text{poly}(1/\epsilon, 1/\delta, \text{size}(x), \text{size}(c))$ ，使得当训练数据数量 $m \geq \text{poly}(\dots)$ 时，算法能从 $\mathcal{H}$ 中辨识 c ，则称 $\mathcal{C}$ 是 **PAC 可学习的**。

样本复杂度：满足上述条件的最小 m 。

5. 不可知 PAC 可学习（第36页）

当目标概念 $c^* \notin \mathcal{H}$ （不可分情形）时，无法学到 c^* 的 ϵ 近似。此时转为 **不可知 PAC 可学习**：希望算法输出的 h 满足

$$P[E(h) - \min_{h' \in \mathcal{H}} E(h') \leq \epsilon] \geq 1 - \delta$$

即：学到的是 $\mathcal{H}$ 中**最优假设**的 ϵ 近似。

6. 轴平行矩形案例（第37-41页）

用轴平行矩形例子直观说明 PAC 学习：一个简单算法——输出包含 D 中所有正例的最小矩形 R^D 。可以证明：

$$P_{D \sim \mathcal{D}^m}[E(R^D) \leq \epsilon] \geq 1 - 4e^{-m\epsilon/4}$$

当 $m \geq (4/\epsilon) \cdot \ln(4/\delta)$ 时， $P[E(R^D) \leq \epsilon] \geq 1 - \delta$ ——满足 PAC 可学习条件。

十八、复杂度——VC 维与 Rademacher 复杂度（第二章 第42-56页）

1. 为什么需要复杂度度量

假设空间 $\mathcal{H}$ 越复杂，找到目标概念的难度越大。需要量化假设空间的复杂度：

- **有限 $\mathcal{H}$** : 直接用假设数目 $|\mathcal{H}|$
- **无限 $\mathcal{H}$** : 用 VC 维（简单但粗糙）或 Rademacher 复杂度（精确但难算）

2. VC 维的定义（第43-46页）

限制 Restriction: $\mathcal{H}$ 在数据集 D 上的限制 $\mathcal{H}|_D = \{(h(x_1), \dots, h(x_m)) \mid h \in \mathcal{H}\}$

增长函数: $\Pi_{\mathcal{H}}(m) = \max\{|\mathcal{H}|_D| \mid D \subseteq \mathcal{X}, |D|=m\}$ ($\mathcal{H}$ 对 m 个数据最多能给出多少种不同标记结果)

打散 Shatter: 若 $|\mathcal{H}|_D| = 2^m$ (实现了所有可能标记结果), 则称 D 被 $\mathcal{H}$ 打散。

VC 维的定义:

$$VC(\mathcal{H}) = \max\{m : \Pi_{\mathcal{H}}(m) = 2^m\}$$

即: 能被 $\mathcal{H}$ 打散的最大样本集大小。

- $VC(\mathcal{H}) = k$ 等价于: 存在大小为 k 的样本集能被 $\mathcal{H}$ 打散, 且任意大小为 $k+1$ 的样本集都不能被 $\mathcal{H}$ 打散
- VC 维与数据分布无关
- 对于有限 $\mathcal{H}$, $VC(\mathcal{H}) \leq \log_2 |\mathcal{H}|$

3. 线性分类器的 VC 维（第47-49页）

- **齐次线性超平面** ($b=0$, 经过原点): VC 维 = d (特征维数)
- **一般线性超平面** ($b \neq 0$): VC 维 = $d + 1$

证明思路 (d 维可被打散): 取 d 个单位向量 $\{e_1, \dots, e_d\}$ 作为数据集。要实现任意标记结果 $(y_1, \dots, y_d)$, 只需取 $w \cdot y = (y_1, \dots, y_d)$ 即可—— $w \cdot y^T e_i = y_i$ 。

证明思路 (d+1 维不能被打散): 任意 $d+1$ 个向量必然线性相关, 存在不全为零的系数 $\{a_i\}$, 可通过矛盾证明无法同时正确标记这 $d+1$ 个点。

4. 神经网络的 VC 维（第50页）

记 N 为网络的参数量, 其 VC 维上界为 $O(N \log N)$ 。参数量越大 $\rightarrow$ VC 维越高 $\rightarrow$ 模型越复杂。

5. Rademacher 复杂度（第51-56页）

动机: VC 维不考虑具体学习问题的数据分布特点, 刻画相对粗糙。Rademacher 复杂度结合了具体数据特点。

经验 Rademacher 复杂度:

$$\hat{\mathfrak{R}}_D(\mathcal{F}) = \mathbb{E}_{\sigma} \left[\sup_{f \in \mathcal{F}} \frac{1}{m} \sum_{i=1}^m \sigma_i f(x_i) \right]$$

其中 σ_i 是 Rademacher 随机变量 (以各 0.5 概率取 -1 或 +1)。

期望 Rademacher 复杂度: $\mathfrak{R}_m(\mathcal{F}) = \mathbb{E}_{D \sim \mathcal{D}^m} [\hat{\mathfrak{R}}_D(\mathcal{F})]$

直观理解: 用随机噪声 σ_i 替代真实标签 y_i , 衡量 $\mathcal{F}$ 能否「拟合」纯随机噪声的能力。复杂度高 $\rightarrow$ 连随机噪声都能拟合 $\rightarrow$ 泛化差。

线性超平面的 Rademacher 界: 假设 $\|x\| \leq r, \|w\| \leq \Lambda$, 则 $\mathfrak{R}_D(\mathcal{F}) \leq r^2 \Lambda^2 / m$ ——反映了数据相关特性。

6. VC 维 vs Rademacher 复杂度对比

维度	VC 维	Rademacher 复杂度
数据依赖	不依赖	依赖数据分布
计算难度	简单	更复杂
精度	相对粗糙	更精确
适用范围	二分类 $\mathcal{H}$	任意实值函数空间 $\mathcal{F}$

十九、泛化性（第二章 第57-65页）

1. 泛化误差界

PAC 理论的核心结论——泛化误差上界依赖于**假设空间大小**和**数据集大小**。

2. 有限可分情形（第57-58页）

目标概念 $c \in \mathcal{H}$ 且 $|\mathcal{H}|$ 有上界。

核心结论：当 $m \geq (1/\epsilon) \cdot \ln(|\mathcal{H}|/\delta)$ 时，经验误差为 0 的假设 h 满足 $P[E(h) \leq \epsilon] \geq 1 - \delta$ 。

证明思路：泛化误差 $> \epsilon$ 且经验误差为 0 的假设，其所有 m 次预测碰巧全部正确的概率 $\leq (1-\epsilon)^m$ 。取并集界得 $|\mathcal{H}| \cdot (1-\epsilon)^m \leq |\mathcal{H}| \cdot e^{-\epsilon m} \leq \delta$ 。

泛化上界以 **$O(1/m)$** 速率收敛——**数据集越大，泛化误差界越小**。

3. 有限不可分情形（第59-60页）

目标概念 $c \notin \mathcal{H}$ ($|\mathcal{H}|$ 有上界但不可分)。

核心结论：以至少 $1-\delta$ 的概率，对所有 $h \in \mathcal{H}$ ：

$$E(h) - \hat{E}(h) \leq \sqrt{\frac{1}{2m} \ln \frac{2|\mathcal{H}|}{\delta}}$$

收敛速率为 **$O(1/\sqrt{m})$** ——比可分情形的 $O(1/m)$ 更慢，需要更多数据。

4. 无限假设空间——VC 维泛化界（第61页）

当 $\mathcal{H}$ 的 VC 维为 k 且 $m > k$ 时，以至少 $1-\delta$ 的概率：

$$E(h) - \hat{E}(h) \leq \sqrt{\frac{8k \ln(2em/k) + 8 \ln(4/\delta)}{m}}$$

收敛速率为 **$O(\sqrt{(k \cdot \ln(m/k))/m})$** ——受 VC 维影响，速度比有限不可分情形更慢。

5. 无限假设空间——Rademacher 泛化界（第62页）

对于实值函数空间 $\mathcal{F}: \mathcal{X} \mapsto [0,1]$ ，以至少 $1-\delta$ 的概率对所有 $f \in \mathcal{F}$ ：

$$\mathbb{E}[f(h)] \leq \frac{1}{m} \sum_{i=1}^m f(h(x_i)) + 2\mathfrak{R}_m(\mathcal{F}) + \sqrt{\frac{\ln(1/\delta)}{2m}}$$

以线性超平面为例，收敛速率为 **$O(1/\sqrt{m})$** ——优于 VC 维的分析结果。

二十、经验风险最小化原则（第二章 第63-65页）

ERM 原则：学习算法输出假设空间 $\mathcal{H}$ 中具有**最小经验误差**的假设：

$$h^* \in \arg \min_{h \in \mathcal{H}} \hat{E}(h)$$

ERM 的理论保证：令 g 表示 $\mathcal{H}$ 中具有最小泛化误差的假设。结合 Hoeffding 不等式和 VC 维泛化界可证明：以至少 $1-\delta$ 的概率，ERM 找到的 h^* 满足 $E(h^*) - E(g) \leq \epsilon$ 。

即 ERM 原则找到的假设，以大概率是假设空间内最优假设的 ϵ 近似。

二十一、过拟合和欠拟合（第二章 第66-68页）

定义

- **欠拟合 Underfitting：**模型不能在训练集上获得足够低的误差（容量不足）
- **过拟合 Overfitting：**训练误差很小，但泛化误差（测试误差）很大（容量过大、记忆训练数据）

容量 Capacity 与误差的关系

以多项式回归为例： $f(x) = b + \sum_{i=1}^k w_i x^i$

当 k 太小（容量不足）→ 欠拟合。当 k 太大（容量过大）→ 过拟合。

奥卡姆剃刀原则：在同样能解释已知观测现象的假设中，选择「最简单」的那一个。

没有免费午餐定理 No Free Lunch Theorem

核心思想：**没有任何一个算法在所有问题上都优于其他算法。** 算法的好坏取决于它与具体问题的匹配程度。一种算法在一类问题上表现好，在另一类问题上必然表现差。

二十二、正则化（第三章补充）

岭回归 Ridge Regression（L2 正则化）

在标准线性回归的 MSE 损失上增加 L2 惩罚项：

$$\min_w \|y - Xw\|^2 + \lambda \|w\|_2^2$$

其中 λ 控制正则化强度。 λ 越大，权重 w 越趋向 0。L2 正则化鼓励所有权重都小但不为零。

Lasso 回归（L1 正则化）

$$\min_w \|y - Xw\|^2 + \lambda \|w\|_1$$

L1 正则化倾向于产生**稀疏解**——一部分权重被精确压缩到 0，实现自动特征选择。

二十三、监督学习（第三章 第10-24页）

1. 监督学习定义

给定数据样本属性含特征 x 和标签 y ，学习从特征到标签的映射。典型案例：手写数字识别（分类）、连续值预测（回归）、人脸检测。

2. 二分类（第11-14页）

问题设定： $D = \{(x_i, y_i)\}$, $x_i \in \mathbb{R}^d$, $y_i \in \{-1, +1\}$ 。

线性分类器：超平面 $H = \{x : w^T x + b = 0\}$ 。 $w^T x + b > 0$ 预测为正类，反之为负类。

0-1 损失： $\ell(x, y) = \mathbb{I}[y \cdot (w^T x + b) \leq 0]$ （不可导，实践中不用）

逻辑回归 **Logistic Regression**——用 Sigmoid 函数将输出映射为概率：

$$p(y|x) = \frac{1}{1 + \exp(-y(w^T x + b))}$$

对数似然：最大化 $\sum \log p(y_i | x_i)$

逻辑损失 **Logit Loss**: $\ell_{LR}(x, y) = \log(1 + \exp(-y \cdot (w^T x + b)))$ （该函数凸且平滑，可通过梯度下降高效优化）

3. 多分类（第15-19页）

问题设定： $D = \{(x_i, y_i)\}$, $y_i \in \{1, 2, \dots, k\}$ 。

Softmax 多分类逻辑回归：为每个类别 c 维护一个线性超平面 $w_c^T x + b_c$

$$P(y = c|x) = \frac{\exp(w_c^T x + b_c)}{\sum_{c'=1}^k \exp(w_{c'}^T x + b_{c'})}$$

多分类逻辑损失 **Cross-Entropy**:

$$\ell(x, y) = -\log \frac{\exp(w_y^T x + b_y)}{\sum_{c=1}^k \exp(w_c^T x + b_c)}$$

从多分类到二分类的转换：

- **一对多 OvR**：训练 k 个二分类器（第 i 个区分第 i 类 vs 其他），投票决定最终类别。缺点：每分类器需处理全部数据，可能面临类别不平衡
- **一对一 OvO**：训练 $C(k, 2)$ 个二分类器（每对类别一个），投票决定。缺点：不适用于数据量少的情况

4. 罕见类与类别不平衡（第22-24页）

问题：当类别分布不平衡时（如 99% 正常、1% 异常），简单准确率无效——全猜「正常」即可 99% 准确率。

解决思路：提高罕见类的重要性（赋予更高的错分代价）。

- **样本重加权**：给不同类别样本不同的错分代价 $C(y_i)$
- **样本重采样**：对罕见类过采样，对常规类降采样
- **单边选择**：保留所有罕见类样本，仅采样一部分常规类样本
- ⚠ 过采样可能导致数据重复 → 过拟合

SMOTE 方法（合成过采样）：对每个罕见样本，找到其 K 个最近同类邻居，在样本连线上生成新的合成样本——避免简单复制导致的过拟合。

二十四、无监督学习（第三章 第26-43页）

1. 无监督学习定义

数据样本只包含特征 x ，**无人工标注标签 y** 。模型需要从数据内在结构或分布中学习。主要类型：聚类、降维、自监督学习。

2. K-Means 聚类算法（第28-32页）

损失函数：最小化所有样本到其所属簇中心的距离平方和。

算法流程：

1. 随机初始化 k 个簇中心
2. 分配步骤：将每个样本分配到最近的簇中心
3. 更新步骤：将每个簇中心更新为该簇所有样本的均值
4. 重复 2-3 直到收敛

收敛保证：算法的每一步优化都不会增加损失（分配步和更新步均单调不增）→ 保证收敛（到局部最优）。

注意：糟糕的初始化可能导致收敛到较差的局部最优（常需多次随机初始化选最优结果）。

3. PCA 主成分分析（第34-42页）

动机：不同维度特征之间存在相关性时，是否仍需保留所有特征？PCA 通过坐标轴旋转找到方差最大的投影方向，实现降维。

数学推导——一维 PCA：

目的是找单位向量 w ，使数据投影到 w 方向后的方差最大：

$$\max_{w \in \mathbb{R}^d} w^T C w, \quad s.t. \|w\|^2 = 1$$

其中 $C = (1/m) \sum (x_i - \bar{x})(x_i - \bar{x})^T$ 是数据的协方差矩阵。

求解：构造拉格朗日函数 → 对 w 求导 → $Cw = \lambda w$ → w 是 C 的**特征向量**， λ 是**特征值**。

最优解： $w_1 = C$ 的**最大特征值**对应的特征向量（即第一主成分）。

k 维 PCA：

$$\max_{W \in \mathbb{R}^{d \times k}} \text{trace}(W^T C W), \quad s.t. W^T W = I$$

解： $W = [w_1, \dots, w_k]$ ，其中 w_i 是 C 的第 i 大特征值对应的特征向量。

信息量（保留方差的占比）：
$$\frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^d \lambda_i}$$

其中 λ_i 是 C 的第 i 大特征值（也等于第 i 个新坐标的方差）。

4. 自监督学习（第43页）

自监督学习是深度学习时代的主流无监督学习方法——通过数据自身特性构造「伪标签」。典型例子：对比学习（SimCLR）、下一词预测（GPT）——无需人工标注，从数据自身挖掘监督信号。

附录：核心公式速查表

激活函数

函数	公式	导数	特点
Sigmoid	$\sigma(x) = 1/(1+e^{-x})$	$\sigma'(x) = \sigma(1-\sigma) \leq 0.25$	易梯度消失
Tanh	$\tanh(x)$	$1-\tanh^2(x)$	零中心化
ReLU	$\max(0,x)$	$1 \ (x>0) / 0 \ (x\leq 0)$	正轴不消失, Dying ReLU
Leaky ReLU	$\max(\alpha x,x)$	$1 \ (x>0) / \alpha \ (x\leq 0)$	解决Dying ReLU

损失函数

任务	损失函数	公式
回归	MSE	$(1/n)\sum // y-\hat{y} //^2$
回归	MAE	$(1/n)\sum$
多分类	交叉熵	$-\sum y_c \log \hat{y}_c$

归一化

方法	公式	归一化维度
Batch Norm	$\gamma \cdot (x-\mu_B)/\sigma_B + \beta$	跨 batch
Layer Norm	$\gamma \cdot (x-\mu_L)/\sigma_L + \beta$	跨 feature
Instance Norm	$\gamma \cdot (x-\mu_{IN})/\sigma_{IN} + \beta$	单样本单通道
Group Norm	$\gamma \cdot (x-\mu_{GN})/\sigma_{GN} + \beta$	通道分组后

初始化

方法	方差	适用激活函数
Xavier	$2/(d_{\ell} + d_{\ell-1})$	tanh / sigmoid
He	$2/d_{\ell-1}$	ReLU 及其变体

优化器

方法	核心机制	关键参数
SGD	$w -= \eta \cdot g$	η
SGD+Momentum	$v = \rho v + (1-\rho)g; w -= \eta v$	η, ρ

方法	核心机制	关键参数
AdaGrad	$G += g^2; w -= \eta g / \sqrt{G}$	η
RMSProp	$G = \rho G + (1-\rho)g^2$	η, ρ
Adam	$m = \beta_1 m + (1-\beta_1)g; v = \beta_2 v + (1-\beta_2)g^2$	$\eta, \beta_1=0.9, \beta_2=0.999$
AdamW	Adam + 去耦权重衰减	$\eta, \beta_1, \beta_2, \lambda$

Dropout

阶段	操作	公式
训练	随机丢弃	$\tilde{a} = r \odot a, r \sim \text{Bernoulli}(p)$
测试	缩放对齐	$a_{\text{test}} = p \cdot a$

文档完成时间：2026 年 6 月

来源：第四章「神经网络基础（一）」101页 + 第五章「神经网络基础（二）」95页

缺失：第一至第三章（机器学习基础部分）——请提供对应 PPT 文件路径以补充

深度视觉模型（第六章 & 第七章）—— 考试知识点总结

根据考试范围整理：卷积计算、mAP/IoU 算子、R-CNN → YOLO 核心改进、ResNet 架构、上采样技术

一、卷积的所有计算

1. 单步卷积计算

卷积核在输入图像上滑动，每次取一个局部区域，做逐元素相乘再求和。

数值示例：

输入 (3×3)：

26	120	126
56	76	99
46	106	101

卷积核 (2×2)：

1	0
0	1

核在左上角时的计算：

$26 \times 1 + 120 \times 0 + 56 \times 0 + 76 \times 1 = 26 + 0 + 0 + 76 = 102$

核向右滑动一格：

120×1 + 126×0 + 76×0 + 99×1 = 219

核向下滑动一格：

56×1 + 76×0 + 46×0 + 106×1 = 162

核再向右滑动：

76×1 + 99×0 + 106×0 + 101×1 = 177

最终输出（2×2 特征图）：

102	219
162	177

2. 多通道卷积（RGB 三通道）

输入形状：C_in × H × W（如 RGB = 3 × 224 × 224）

卷积核深度必须等于输入通道数。一个卷积核的形状是 C_in × K_H × K_W。

计算：卷积核的三个通道分别和 R、G、B 三个通道做二维卷积 → 三个中间结果累加 → 一个输出通道。

要得到 C_out 个输出通道 → 用 C_out 个独立的卷积核。每个核产出一个通道，堆叠起来 = C_out × H' × W'。

每个输出通道的计算：

$$y_{c_{out},i,j} = \sum_{c_{in}=0}^{C_{in}-1} \sum_{p=0}^{K-1} \sum_{q=0}^{K-1} W_{c_{out},c_{in},p,q} \cdot x_{c_{in},i+p,j+q} + b_{c_{out}}$$

3. 输出尺寸公式（必须会手算）

$$H' = \frac{H-K+2P}{S} + 1$$

$$W' = \frac{W-K+2P}{S} + 1$$

符号	含义
H, W	输入的高度和宽度
K	卷积核尺寸（假设方形 K×K）
P	Padding（每边补几圈 0）
S	Stride（步长）

计算示例：

输入 32×32，K=5，P=2，S=1：

H' = (32 - 5 + 4) / 1 + 1 = 32 → 输出 32×32

输入 224×224，K=11，P=0，S=4：

H' = (224 - 11 + 0) / 4 + 1 = 54.25 + 1 ≈ 55

⚠ 结果必须是正整数，否则卷积核无法整齐滑过

4. 参数量计算

卷积层权重参数量：

$$C_{out} \times C_{in} \times K_H \times K_W$$

卷积层偏置参数量： C_{out} （每个输出通道一个偏置）

全连接层参数量：

输入神经元数 × 输出神经元数 + 输出神经元数 （偏置）

计算示例：

输入 224×224×3 （150,528 维），输出 1000：

全连接： $150,528 \times 1,000 + 1,000 \approx 1.5$ 亿

卷积 （1000 个 3×3×3 核）： $3 \times 3 \times 3 \times 1,000 + 1,000 = 28,000$

差距约 5500 倍

5. 感受野堆叠公式

$$R = 1 + L \times (K - 1)$$

其中 L 是层数，K 是每层卷积核尺寸。

堆叠方式	感受野	参数量（输入输出通道均为 C）
1 层 3×3	3×3	$9C^2$
2 层 3×3	5×5	$18C^2$
3 层 3×3	7×7	$27C^2$
1 层 7×7	7×7	$49C^2$

结论： 3 个 3×3 和 1 个 7×7 感受野相同，但参数量少 45%，且中间多了 2 次 ReLU。

计算示例： 输入 224×224，只用 3×3 卷积不用池化，需要多少层覆盖全图？

$$1 + L \times (3 - 1) = 224 \rightarrow L = 111.5 \rightarrow \text{至少 112 层}$$

6. 计算量（FLOPs）

标准卷积计算量：

$$K^2 \times C_{in} \times C_{out} \times H' \times W'$$

深度可分离卷积（MobileNet V1）：

$$\text{深度卷积： } K^2 \times C_{in} \times H' \times W'$$

$$\text{逐点卷积（1×1）： } C_{in} \times C_{out} \times H' \times W'$$

计算量比： 约为 $1 / C_{out} + 1 / K^2$ （通常为标准卷积的 1/8 ~ 1/9）

7. 池化层输出尺寸公式

$$H' = \frac{H-K}{S} + 1$$

池化层没有可学习参数。

8. 1×1 卷积

对每个空间位置 (h,w) 的 C_in 维向量做线性变换 → 输出 C_out 维向量。

$$y_{h,w} = W_{1 \times 1} \cdot x_{h,w} + b$$

参数量 = C_in × C_out (每个 1×1 核)，忽略空间尺寸。

Bottleneck 计算省参数量 (ResNet-50)：

直接 3×3 (256→256)：3² × 256² = 589,824

Bottleneck：1²×256×64 + 3²×64² + 1²×64×256 = 69,632 (省 88%)

二、mAP、IoU 等算子的计算方法

1. IoU (Intersection over Union，交并比)

定义：预测框和真实框的交集面积 / 并集面积。

$$IoU = \frac{|A \cap B|}{|A \cup B|} = \frac{\text{交集面积}}{A\text{面积} + B\text{面积} - \text{交集面积}}$$

取值：0 (完全不重叠) ~ 1 (完全重叠)

Faster R-CNN 中 IoU 的阈值约定：

IoU 范围	样本类型	用途
IoU > 0.7	正样本 (前景，有物体)	参与分类和回归训练
0.3 < IoU < 0.7	忽略样本	不参与训练
IoU < 0.3	负样本 (背景)	参与分类训练

2. Precision 和 Recall

混淆矩阵：

	预测为正	预测为负
真实为正	TP (真正例)	FN (假负例 / 漏检)
真实为负	FP (假正例 / 误检)	TN (真负例)

$$Precision = \frac{TP}{TP+FP}$$

(所有被检测为「物体」的框中，有多少是对的)

$$Recall = \frac{TP}{TP+FN}$$

(所有真正的物体中，有多少被检测出来了)

3. AP (Average Precision, 平均精度)

单类的平均精度。以 Recall 为横轴、Precision 为纵轴画 PR 曲线，AP = PR 曲线下面积。

计算步骤：

- 对一个类的所有检测框按置信度降序排列
- 从高到低逐框计算当前的 Precision 和 Recall
- 用 11 点插值法或全点积分法计算 PR 曲线下面积

$$AP = \sum_{k=1}^N P(k) \times \Delta R(k)$$

4. mAP (mean Average Precision)

所有类别的 AP 取平均：

$$mAP = \frac{1}{C} \sum_{c=1}^C AP_c$$

其中 C 是类别数。

mAP@0.5：IoU 阈值为 0.5 时的 mAP（较宽松）

mAP@0.5:0.95：IoU 阈值从 0.5 到 0.95，步长 0.05，取平均（COCO 标准，更严格）

5. mIoU (mean IoU, 语义分割评估指标)

所有类别的 IoU 取平均：

$$mIoU = \frac{1}{C} \sum_{c=1}^C \frac{TP_c}{TP_c + FP_c + FN_c}$$

其中 TP_c、FP_c、FN_c 是在像素级别计算的。

6. NMS (Non-Maximum Suppression, 非极大值抑制)

用于去除同一物体的重复检测框。**不直接参与 mAP 计算，但是 mAP 评估前的必要后处理步骤。**

算法步骤：

- 将所有框按置信度降序排列
- 取最高分框 A，计算 A 和其余所有框的 IoU
- 将 IoU > 阈值（如 0.5）的框全部删除
- 对剩余框重复以上步骤，直到没有框剩下
- 最终保留下来的就是不重叠的、各有最高分的框

三、R-CNN 到 YOLO 的核心改进

整体脉络



1. R-CNN

流程：

1. Selective Search 在原图上生成约 2000 个候选区域 (RoI)
2. 每个 RoI 缩放到固定尺寸 (如 224×224)
3. 每个 RoI 独立通过 CNN 提取特征
4. 用 SVM 对每个 RoI 的特征做分类
5. 用线性回归器对每个 RoI 的框做精调 (预测 dx, dy, dw, dh)

问题：每张图 2000 次 CNN 前向传播，大量 RoI 空间重叠造成重复计算；CNN、SVM、回归器分开训练，不是端到端

2. Fast R-CNN

核心改进：

- **共享特征图**：整张图只走一次 CNN，得到一张共享特征图。所有 RoI 从特征图上取特征
- **RoI Pooling**：将任意尺寸的 RoI 特征块均匀分成固定网格 (如 7×7)，每个格内 Max Pooling → 固定尺寸输出
- **端到端训练**：CNN + 分类头 (Softmax) + 回归头，联合训练
- 速度：比 R-CNN 训练快约 9 倍，推理快约 200 倍

遗留问题：Selective Search 仍在 CPU 上运行 (~2 秒/图)，成为整个系统的瓶颈

3. Faster R-CNN

核心改进：引入 RPN (Region Proposal Network) 替代 Selective Search

RPN 和检测网络共享同一个 CNN 骨干网络提取的特征图。RPN 直接在特征图上预测候选区域。

锚点框 Anchor Boxes：

- 在特征图的每个空间位置预设 K 个锚点框 (如 3 种尺度 × 3 种长宽比 = 9 个)
- 锚点框覆盖了物体可能的各种大小和形状

RPN 的两个任务：

- 二分类：对每个锚点框判断是「前景」(有物体) 还是「背景」
- 回归：对前景锚点框预测 4 个修正量 (dx, dy, dw, dh)

推理流程：

- RPN 对所有锚点框按前景得分排序 → 取前约 300 个作为 Proposal
- 检测网络对这 300 个 Proposal 做精细分类和回归

意义：实现了**完全端到端、全 GPU 上运行**的目标检测

总损失：

$$L = L_{cls} \text{ (分类损失/交叉熵)} + \lambda \cdot L_{reg} \text{ (定位损失/平滑 L1)}$$

4. YOLO (You Only Look Once)

核心理念：一次前向传播，直接输出所有检测结果。不再有「先提议再分类」的两步走。

工作流程：

- 特征图尺寸 $S \times S$ (如 7×7)。物体的中心落在哪个空间位置 → 由那个位置的特征向量负责预测这个物体
- 每个空间位置输出 B 个边界框 (每个框 5 个值: $x, y, w, h, confidence$) + C 个类别概率
- x, y = 相对于当前空间位置的偏移量 (格内坐标转化为绝对坐标)
- w, h = 整个框的宽高 (用原图比例表示)
- 总输出维度: $S \times S \times (B \times 5 + C)$

后处理：

- 置信度筛选: $confidence \times P(class) < \text{阈值的框全部丢弃}$
- NMS: 对同一类别高度重叠的框，只保留得分最高的那个

优势：速度快，适合实时应用 (单次前向传播出所有结果)

局限：

- 对小物体和密集物体的效果不如两阶段方法
- $S \times S$ 的分辨率限制了精确定位能力

5. YOLO 相对 R-CNN 系列的核心改进总结

维度	R-CNN 系列	YOLO
检测方式	先提议 → 后分类 (两阶段)	一次前向全出 (单阶段)
候选区域	外部算法或 RPN 网络生成	不需要单独的提议阶段
速度	相对慢 (尤其是 R-CNN)	快，实时
每个区域的处理	每个提议区域独立分类	全图统一处理
输出结构	可变数量 (取决于提议数量)	固定维度 $S \times S \times (B \times 5 + C)$
小物体检测	较好 (RPN 多尺度锚点)	相对弱 (单尺度特征图)

6. Fast R-CNN 到 Faster R-CNN 到 YOLO 的三步进化

Fast R-CNN（共享卷积，但区域提议靠外部 Selective Search）

↓ 把区域提议也纳入网络

Faster R-CNN（RPN 网络替代 Selective Search，全 GPU 端到端）

↓ 把「提议 + 分类」两步合成一步

YOLO（直接在特征图上密集预测，不需要显式的提议阶段）

四、ResNet 架构

1. 退化问题

在 ResNet 之前，人们发现直接堆更深层 → **训练误差反而上升**（不是过拟合）。

理论上，56 层网络的后面 36 层只要学恒等映射 $H(x) = x$ ，就至少不比 20 层差。但实际上**非线性层很难学会恒等映射**。

2. 残差连接：核心公式

$$y = F(x) + x$$

- 不直接学目标映射 $H(x)$ ，而是学残差 $F(x) = H(x) - x$
- 如果最优映射就是恒等映射， $F(x)$ 只需趋近于 0
- 推权重趋向 0 比学精确的恒等映射容易得多

3. 梯度流动

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \left(1 + \frac{\partial F}{\partial x}\right)$$

关键在那个 +1。即使 $F(x)$ 内部的梯度衰减为 0，梯度仍能通过跳跃连接无损传回。**这是 ResNet 能训练 152 层甚至 1000+ 层的本质原因。**

4. 残差块结构

Basic Block（ResNet-18/34 用）：

输入 → 3×3 Conv → BN → ReLU → 3×3 Conv → BN → + 输入 → ReLU

Bottleneck Block（ResNet-50/101/152 用）：

输入 → 1×1 Conv（降维，如 256→64）→ BN → ReLU → 3×3 Conv（空间卷积，64→64）→ BN → ReLU → 1×1 Conv（恢复维度，64→256）→ BN → + 输入 → ReLU

Bottleneck 参数量对比（输入输出均为 256 通道）：

直接 3×3：3²×256² = 589,824

Bottleneck：1²×256×64 + 3²×64² + 1²×64×256 = 69,632（省 88%）

5. ResNet 整体架构

输入 224×224×3

→ Conv 7×7, S=2, 64 通道

→ MaxPool 3×3, S=2

→ [残差块 ×3] (64 通道, 不改变尺寸)

→ [残差块 ×4] (128 通道, 第一个块 stride=2 下采样)

→ [残差块 ×6] (256 通道)

→ [残差块 ×3] (512 通道)

→ Global Average Pool

→ FC(512 → 1000) + Softmax

特点：

- 用 stride=2 的卷积替代池化层做下采样
- 网络末端用 Global Average Pooling 替代大型全连接层
- 只有最后一层才是全连接层

6. 维度不匹配的解决方案

当 stride=2 或通道数改变时，F(x) 和 x 的形状不一致，不能直接相加。

Projection Shortcut (1×1 投影，最常用)：

$$y = F(x) + W_s x$$

在跳跃连接上用 一个 1×1 卷积 (stride 和输出尺寸与 F(x) 匹配)，将 x 投影到正确的维度和尺寸。

Zero Padding：对缺失的维度填零 (更简单但效果稍差)。

7. ResNet 关键数字

模型	层数	残差块类型	参数量 (约)
ResNet-18	18	Basic	11.7M
ResNet-34	34	Basic	21.8M
ResNet-50	50	Bottleneck	25.6M
ResNet-101	101	Bottleneck	44.5M
ResNet-152	152	Bottleneck	60.2M

里程碑成就：ResNet-152 ImageNet Top-5 错误率 3.57%，首次超越人类水平 (约 5.1%)。

论文引用 20 万+ 次，残差连接成为所有深度架构的标配。

五、上采样技术

为什么语义分割需要上采样

编码器通过反复下采样（池化 / stride=2 卷积）压缩空间尺寸以提取语义信息，特征图变小。但语义分割要求**输出和输入同样大小的像素级预测图**。需要把特征图从小放大回去。

5 种上采样方法

1. 最近邻插值（Nearest Neighbor Interpolation）

做法：目标像素的值 = 离它最近的源像素的值。

7×7 → 14×14 的例子：每个新像素直接「抄」最近老像素的值。

优点：计算极快。

缺点：输出有锯齿，边界粗糙。

2. 双线性插值（Bilinear Interpolation）

做法：取目标像素周围 4 个最近的源像素，按距离加权平均。

两步走：

- 水平方向做两次线性插值
- 垂直方向再做一次

公式（简化）：设目标点离 4 个源像素的距离为 d_1, d_2, d_3, d_4 ，则：

$$y = \frac{\sum (1-d_i) \cdot x_i}{\sum (1-d_i)}$$

优点：比最近邻平滑，计算量适中。

缺点：手工规则，不可学习。

3. Max Unpooling

做法：做 Max Pooling 时记录最大值在 2×2 窗口中的位置；上采样时只把值放回原位置，其余位置填 0。

Max Pooling（前向时同步记录位置）：

2×2 窗口 [12, 20; 8, 12]，最大值 20，位置 (0,1) → 记录

Max Unpooling（上采样时）：

把值 20 放回 (0,1) 位置，其他 3 个位置填 0

优点：比最近邻和双线性保留了更多空间结构信息。

缺点：不可学习，且除最大值外的其他激活信息全丢失。

4. 转置卷积 (Transposed Convolution / Deconvolution)

做法：每个输入像素通过卷积核「扩散」成 $K \times K$ 的输出区域。stride > 1 时，输出像素之间插入 0 来扩大空间。

- stride=2 时输入一个值 → 输出该值被放大并「扩散」至输出的 3×3 区域
- 相邻像素扩散区域重叠时 → 对应位置求和
- 卷积核权重可学习

参数量： $C_{in} \times C_{out} \times K_H \times K_W$ (和标准卷积相同)

优点：可学习，网络自己找到最优上采样方式。

⚠️ **棋盘效应 Checkerboard Artifact**：当 stride 和 kernel size 不匹配时，输出出现规则的格状图案。可用 kernel size 能被 stride 整除的方法缓解 (如 $K=4, S=2$)。

5. 反池化 (Unpooling / 0-填充)

做法：每个值复制到扩大后的对应位置，其他位置填 0。比 Max Unpooling 简单——不记录位置信息。

优点：简单直接。

缺点：精度较低。

语义分割各模型中的上采样方式

模型	上采样方式
FCN	转置卷积 (可学习的上采样)
U-Net	转置卷积或双线性插值 + 卷积
DeepLab v1/v2	双线性插值 (直接放大预测图)
DeepLab v3+	双线性插值 + 解码器特征融合
SegFormer	MLP + 双线性插值 (模型内部处理, 简单高效)
SAM	转置卷积 (掩码上采样到原图尺寸)

上采样在编码器-解码器架构中的位置

编码器 (下采样通道) : $224 \times 224 \rightarrow 112 \rightarrow 56 \rightarrow 28 \rightarrow 14 \rightarrow 7$

↓

解码器 (上采样通道) : $7 \rightarrow 14 \rightarrow 28 \rightarrow 56 \rightarrow 112 \rightarrow 224$

(通过跳跃连接融合编码器对应层的特征)

考试重点速查表

卷积计算

公式	用途
$H' = (H - K + 2P) / S + 1$	卷积输出尺寸
$H' = (H - K) / S + 1$	池化输出尺寸
$C_{out} \times C_{in} \times K_H \times K_W$	卷积权重参数量
$R = 1 + L \times (K - 1)$	感受野大小
3 个 3×3 vs 1 个 7×7	27C² vs 49C²，省 45%，多 2 次 ReLU

评估指标

指标	定义	公式
IoU	交并比	交集 / 并集
mAP	各类 AP 均值	$\text{sum}(\text{AP}_c) / C$
mIoU	各类 IoU 均值	$\text{sum}(\text{IoU}_c) / C$

R-CNN → YOLO 演进要点

R-CNN：Selective Search + 逐 RoI CNN → 太慢（2000 次 CNN/图）

Fast R-CNN：共享特征图 + RoI Pooling → 端到端

Faster R-CNN：RPN 替代 Selective Search → 全 GPU

YOLO：单次前向 → 密集预测 → 实时

ResNet 要点

- 公式： $y = F(x) + x$
- 梯度： $\partial L / \partial x = \partial L / \partial y \times (1 + \partial F / \partial x)$ ，永远有 +1
- Bottleneck：1×1 降维 → 3×3 → 1×1 升维，省 88% 计算
- ResNet-152：Top-5 错误率 3.57%，超越人类

上采样要点

方法	可学习	特点
最近邻	否	最快但粗糙
双线性	否	平滑，各向 4 点加权
Max Unpooling	否	保留位置信息

方法	可学习	特点
转置卷积	是	可学习，⚠️ 棋盘效应

期末考试 —— 核心考点详解

基于第八至十二章 PPT 内容精编，覆盖三张图标注的所有考试内容

第一板块：大语言模型架构与 RAG（第十一章 / 第十二章）

一、三种 Transformer 架构

1. 总览

原始 Transformer（2017）是一个完整的 Encoder-Decoder 架构，用于机器翻译。后来的研究者发现这个架构可以被「拆开」使用，衍生出三种变体：

架构	选用部分	注意力掩码	代表模型	擅长
Encoder-only	只用 Encoder	完全注意力（双向，每 Token 看所有 Token）	BERT、RoBERTa	理解任务（分类/情感/实体识别）
Decoder-only	只用 Decoder	下三角因果掩码（单向，只看前不看后）	GPT 系列、LLaMA 系列	生成任务（对话/续写/代码）
Encoder-Decoder	Encoder + Decoder	完全 + 下三角 + 交叉注意力	T5、BART	输入→输出映射（翻译/摘要）

2. Encoder-only 架构

结构：仅使用 Transformer 的编码器部分。包含三层：输入编码（Token 嵌入 + 位置编码）→ 特征编码（多个 Encoder Block 堆叠，每块含双向自注意力 + FFN）→ 任务处理（接分类头输出标签）。

注意力特点：双向完全注意力。每个 Token 在自注意力层可以看到序列中所有其他 Token——包括前面的和后面的。这使得 Encoder 能深度整合完整上下文来理解每个词语的语义。

代表模型——BERT：

- BERT-Base：12 个编码模块，隐藏维 768，注意力头 12，总参数 1.1 亿
- BERT-Large：24 个编码模块，隐藏维 1024，注意力头 16，总参数 3.4 亿
- 预训练数据：BookCorpus（8 亿 Token）+ 英语维基百科（25 亿 Token）≈ 33 亿 Token / ~15GB
- 预训练任务：
 - **MLM（掩码语言建模）：**随机遮挡 15% Token，用双向上下文预测被遮词（完形填空）
 - **NSP（下文预测）：**判断两句是否在原文中相邻

代表模型——RoBERTa：

- 架构与 BERT 完全一致，但训练更充分
- 数据量：15GB → 160GB（增加 CC-News、OpenWebText、Stories）
- 训练步数：100K → 500K
- **动态掩码**：每次加载数据时重新随机生成掩码位置（而非静态固定掩码）
- 移除 NSP 任务

代表模型——ALBERT：

- 参数因子分解：词嵌入矩阵 $V \times H \rightarrow V \times E + E \times H$ ($E \ll H$)，嵌入参数从 2300 万降至 307 万
- 跨层参数共享：所有 Transformer Block 共享同一组参数
- NSP 替换为 SOP（句序预测——判断两句是正序还是倒序）

优势：双向注意力捕捉最丰富的上下文语义，理解任务上极其出色。

局限：没有解码器，不能自回归生成文本；参数量偏小（最大 ~3.5 亿），在生成式 AI 时代被边缘化。

3. Decoder-only 架构

结构：仅使用 Transformer 的解码器部分。因为去掉了 Encoder，同时也去掉了交叉注意力子层，只保留**因果自注意力 + 前馈网络**。

注意力特点：下三角因果掩码。每个 Token 只能看到自己以及前面的 Token，不能看到后面的 Token。数学上通过一个下三角掩码矩阵 + softmax 实现——未来位置的注意力分数被设为 $-\infty$ ，softmax 后权重变为 0。

为什么要因果掩码：保证自回归生成的因果一致性——在预测第 t 个 Token 时，第 t+1 个 Token 还没有被生成出来，所以模型不能「偷看」未来。

代表模型——GPT 系列五个阶段：

阶段	模型	年份	最大参数	数据量	关键突破
初出茅庐	GPT-1	2018	1.17 亿	~5GB	证明单向自回归预训练可行
小有所成	GPT-2	2019	15 亿	40GB	零样本任务能力
崭露头角	GPT-3	2020	1750 亿	1TB	涌现上下文学习 ICL
一鸣惊人	ChatGPT	2022	未公开	未公开	RLHF + LLMaaS 服务模式
大放异彩	GPT-4/4o	2023-24	未公开	未公开	图文双模态 / 降低延迟

GPT-3 的涌现能力——上下文学习 (In-Context Learning, ICL)：

- **Zero-shot**：只给任务的自然语言描述，不给示例，模型直接输出答案
- **Few-shot**：给任务描述 + 几个示例（如 2-3 个问答对），模型参照示例回答新问题
- 不需要梯度更新、不需要微调——纯粹的前向传播即可完成新任务
- 这是「涌现能力」的典型代表——参数增加到一定规模后自然出现，不是编写代码实现的

Representative 模型——LLaMA 系列：

模型	年份	最大参数	数据量	关键改进
LLaMA-1	2023.02	652 亿	~5TB	RoPE / Pre-Norm / SwiGLU
LLaMA-2	2023.07	700 亿	~7TB	+RLHF + 拒绝采样 + GQA
LLaMA-3	2024.04	700 亿	~50TB	数据量 7 倍于上代
LLaMA-3.1	2024.07	4050 亿	~50TB	训练需 3930 万 H100 GPU 小时

LLaMA 的三项架构创新：

- **RoPE（旋转位置编码）**：用旋转矩阵将位置信息注入 Q 和 K 向量，注意力分数天然取决于 Token 间的相对位置差异，长序列泛化性好
- **Pre-Norm**：在每子层输入之前做层正则化（而非传统在输出之后），深层训练更稳定
- **SwiGLU**：替换 ReLU，结合 Swish 平滑非线性和 GLU 门控机制，前馈网络表达力更强
- **GQA（分组查询注意力，LLaMA-2 引入）**：多个 Query 头共享同一组 Key/Value 头，减少 KV 缓存

Decoder-only 成为主流的原因：

- 省掉编码器 → 约一半参数集中用在生成侧 → 同等资源下生成能力更强
- 架构极简 → 分布式扩展更易 → 可堆叠上百层数千亿参数
- 自回归生成天然适合开放域对话、写作、代码生成等主流应用

4. Encoder-Decoder 架构

结构：完整的 Transformer 架构，包含五个部分：

1. 编码端输入嵌入（源序列 Token → 向量 + 位置编码）
2. 编码端特征编码（多个 Encoder Block，双向自注意力 + FFN）
3. 编码器输出（完整上下文向量矩阵，作为解码器的参考）
4. 解码端输入嵌入（目标序列 Token → 向量 + 位置编码）
5. 解码端处理（每层含因果自注意力 + **交叉注意力** + FFN）

三种注意力共存：

- **编码器自注意力**：完全注意力，双向理解源序列
- **解码器自注意力**：下三角因果掩码，保证生成一致
- **交叉注意力**：Q 来自解码器当前层，K 和 V 来自编码器全输出。生成每个新 Token 时都能查阅完整源文语义

代表模型——T5（Text-to-Text Transfer Transformer）：

- 核心思想：所有 NLP 任务统一为「Text → Text」格式
 - 「翻译成英文：今天天气怎么样？」 → 输出英文翻译
 - 「总结以下内容：……」 → 输出摘要
 - 「情感识别：这电影真棒。」 → 输出「积极情感」
- 不需为每个任务设计单独的输出头——全用同一个生成式模型

- 五版本：T5-Small (0.6 亿) → T5-11B (110 亿)
- 预训练任务：**Span Corruption**（遮连续一段多个 Token，而非 BERT 的单个 Token）
- 数据集：C4（Common Crawl 清洗过滤后 750GB）

代表模型——BART：

- 去噪自编码器：输入被破坏的文本 → Encoder 理解噪音 → Decoder 复原原文
- 五项破坏任务：Token 遮挡、Token 删除、句子打乱、文档旋转、文本填充
- 数据量同 RoBERTa（~160GB）

优势：输入和输出的语义映射能力强，适合翻译、摘要等。**劣势：**编解码双份参数开销，计算成本高，同等资源下规模优势不如 Decoder-only。

二、幻觉（Hallucination）

定义

大语言模型生成的内容看似合理，但实际上与事实相悖、逻辑混乱或包含不实信息。

七大来源

训练数据侧（3 个）：

1. **知识过时：**训练数据截止后世界的变化（体育赛事结果、新闻事件）模型不知道。回答时可能「自信地编造」过时信息。
2. **知识边界：**训练数据不可能覆盖人类全部知识。低频专业信息（如考拉基因组有 26000 个基因）如果不在训练数据中或出现极少，模型无法回答。
3. **知识偏差：**互联网文本含大量偏见和不实信息（如「企业家」默认用男性代词），模型不加辨别地学到并复现。

模型本身侧（4 个）：

4. **知识长尾：**低频知识在训练数据中曝光极少（如 G.W.奥尔波特 vs 弗洛伊德），模型记忆不牢，容易混淆为高频相似实体。
5. **曝光偏差（Exposure Bias）：**Teacher Forcing 训练时喂正确上文，推理时喂自己生成的、带有误差的上文。训练和推理的输入分布不同，模型在自推的上文环境中产生偏离。
6. **解码偏差：**采样的随机性（Top-K/Top-P 随机采样）可能偶然输出低概率的不合理 Token，一步偏差被自回归逐级放大。
7. **对齐不当：**RLHF 阶段标注数据如果有偏差（如偏好冗长回答忽略准确性），模型学到「让人类喜欢」而非「说得正确」。

缓解方案——RAG

核心思想：让模型回答前先「查资料」，用外部知识库的最新权威事实约束生成。

三、RAG 架构

RAG 的基本组成

RAG = 知识检索 (Retrieval) + 生成增强 (Generation)

1. **知识检索模块**: 对输入问题编码 → 从外部知识库检索相关文档 → 返回最相关的 Top-K 文档
2. **生成增强模块**: 检索到的文档 + 原始问题 → 拼接成 Prompt → LLM 生成最终答案

RAG 的架构分类——四大类

分类维度: LLM 参数是否更新 (黑盒 vs 白盒)、检索器是否微调。

第一类：黑盒增强——无微调

- LLM 和检索器参数均不更新，直接组合使用
- 代表方法: **In-Context RALM**: 检索文档直接前置到 Prompt 作为上下文
- 优点: 与 LLM 解耦，实现简单，成本最低
- 缺点: 检索器和 LLM 缺乏交互，RAG 效果依赖 LLM 自身指令跟随能力

第二类：黑盒增强——检索器微调

- LLM 参数固定不变，检索器根据 LLM 的反馈信号调整参数
- 代表方法: **REPLUG**: 用 LLM 的困惑度 PPL 作为监督信号训练检索器，使检索器偏向检索能降低 PPL 的文档
- 优点: 成本低，效果良好，不改 LLM
- 缺点: LLM 参数固定，可能无法与检索器完全适配

第三类：白盒增强——仅微调 LLM

- 检索器不变 (预训练好)，LLM 参数根据检索器输出进行更新
- 代表方法: **RETRO**: 在 LLM 中间层通过交叉注意力将检索文档向量动态融入隐藏状态
- 优点: 优化生成能力，更深度利用外部信息
- 缺点: 微调资源需求高，效果依赖检索器性能

第四类：白盒增强——协同微调

- 检索器和 LLM 参数同步更新，互为目标和信号源
- 代表方法: **ATLAS**: 用 KL 散度联合训练检索器和 LLM，使检索器输出的文档分布与 LLM 实际调用的文档贡献分布一致
- 优点: 深度交互，持续优化，性能最佳
- 缺点: 计算成本极高，工程实现复杂

四大架构对比速查表

架构	代表方法	优点	缺点
黑盒-无微调	In-Context RALM	解耦LLM，易实现，成本最低	缺乏交互，效果难保证
黑盒-检索器微调	REPLUG	迎合LLM需求，成本低效果好	LLM固定可能不适配
白盒-仅微调LLM	RETRO	优化生成，更深度利用外部信息	计算成本高/依赖检索器
白盒-协同微调	ATLAS	深度交互，持续优化性能	成本最高/工程最复杂

四、何时增强

核心原则：非必要不增强

如果 LLM 自身内部知识足以正确回答，强制 RAG 反而可能引入噪声（画蛇添足）。

判断方法一：外部观测法

不侵入模型内部参数，通过外部手段判断模型知识状态：

- **Prompt 直问**：设计 Prompt 直接询问模型是否具备某知识。局限——模型容易过度自信（「都懂」）。
- **观察训练数据**：知识在训练数据的出现频率与模型记忆正相关。无法获取训练数据时，用伪数据统计量（如**流行度**）替代——设定维基百科/谷歌某实体访问量阈值来估计模型是否已知。低频=可能不知道=触发 RAG。
- **构造伪训练数据统计量**：流行度 = 该实体在特定环境被广泛关注的频率。越「流行」的知识模型掌握越好。可设定维基百科过去 30 天页面访问量阈值——超过=流行知识（模型大概率已知），低于=非流行知识（模型可能不知→触发 RAG）。

判断方法二：内部观测法

分析 LLM 在生成文本时的内部状态变化来评估知识水平：

- 处理「已知问题」和「未知问题」时，模型的中间层（注意力层、MLP 层）呈现出**不同的激活动态模式**
- 基于这一特性，训练一个**线性探针（Linear Probe）**——一个简单的线性分类器，根据问题对应的内部表示预测该问题是否属于模型已知
- 类似于通过心率、血压、呼吸频率进行测谎——观测内部信号而非询问外在反应

五、何处增强

输入端增强（最主流）

将检索到的外部知识文本与用户查询拼接在 Prompt 中，输入给 LLM。

- **优点**：直观易实现，无需修改模型结构，适用于黑盒 API
- **缺点**：检索文本过长时增加推理计算成本

中间层增强

将外部知识先转换为向量表示，通过交叉注意力（Cross-Attention）融合到模型的隐藏状态中。

- **代表方法：**RETRO——在 LLM 中间层通过交叉注意力将检索文档向量动态融入隐层
- **优点：**更深入影响模型内部表示，减少对输入长度的依赖
- **缺点：**需要对模型结构进行复杂设计和调整，无法用于黑盒模型

输出端增强

先生成初步文本，然后利用外部知识对生成内容进行**后校正**（post-hoc 校准）。

- **代表方法：**RETRO 框架的输出端校正——比对生成内容与检索信息的一致性，动态调整生成结果并矫正潜在错误
- **优点：**确保生成文本与外部知识一致，提高准确性和可靠性
- **缺点：**效果依赖检索知识质量和相关性，校准结果易受不良检索文档影响

三种位置对比速查表

位置	优点	缺点
输入端	直观易实现，无需改结构	长文本增加推理成本
中间层	深度影响模型内部，少受输入长度限制	需复杂设计和调整，不适用黑盒
输出端	确保一致，提高准确可靠性	依赖检索质量和相关性

六、降本增效

问题

检索的外部知识常包含大量原始文本，不经处理直接输入 LLM 会显著增加计算成本。

方案一：去除冗余文本

对检索出的原始文本进行词句过滤，选择出有益于增强生成的部分。

三种级别：

1. **Token 级别：**去除低困惑度 Token（哪些词对 LLM 来说完全可预测、不需外部参考）
2. **全文本级别：**从检出的各文档中提取最重要的核心句
3. **子文本级别：**对文档分割后的子文本打分，删除不必要子文本

代表方法——FIT-RAG：

- 将检索文档划为子文档
- 用预先构建的 LLM 偏好打分器对每个子文本打分
- 筛选出既包含事实信息又符合 LLM 偏好的少量子文本组合
- **效果：**平均压缩约 50% 的输入 Token，回答准确率优于全量基线

方案二：复用计算结果

对计算必需的中间结果（KV-cache）进行缓存复用，避免重复计算。

KV-cache 机制：将每个 Token 生成过程中产生的 Key 和 Value 向量缓存。下次用到相同前文时直接从缓存读取，减少重复前向传播。

RAGCache：RAG 系统专用的多级动态缓存机制，包含三部分：

- KV 张量缓存库
- 缓存检索器
- RAG 控制器

方案三：组合使用

Prompt 压缩 + KV-cache 机制联合使用，在保持高性能的同时显著降低 Token 开销和计算延迟。

第二板块：表征学习（第八章）

七、SimCLR 的网络结构和输入

SimCLR 的整体框架

SimCLR（A Simple Framework for Contrastive Learning of Visual Representations, ICML 2020）是自监督对比学习的里程碑。它**不需要任何人工标签**，通过数据增强自动构建正负样本对来学习图像表征。

输入与数据增强

输入构造方式：

1. 取一个 batch 的 N 张原始图像（无标签）
2. 对每张图像施加**两种不同的**随机数据增强 → 得到 2N 张增强后的图像
3. 同一张原始图像的两个增强版本互为正样本对（共 N 对）
4. batch 内其他 2N-2 张图都是负样本

关键发现——数据增强组合对自监督学习质量起决定性作用：

- **单纯的随机裁剪不够：**模型可以通过颜色分布匹配来「作弊」——判断两个裁剪块是否来自同一张图，无需理解语义内容
- **必须结合颜色抖动：**随机改变亮度、对比度、饱和度、色相——迫使模型学习语义特征而非表面色彩统计
- **典型增强组合：**随机裁剪 + 随机水平翻转 + 颜色抖动 + 高斯模糊

网络结构

SimCLR 包含三个核心组件：

1. 基础编码器 f（Base Encoder）：

- 通常采用 ResNet（如 ResNet-50）作为骨干网络
- 输入：增强后的图像（224×224×3）

- 输出：高维特征向量 h （如 2048 维）
- 负责将原始图像转化为捕捉核心语义内容的特征向量

2. 投影头 g (Projector):

- 一个 2~3 层的 MLP（全连接网络）
- 输入：编码器输出的特征向量 h
- 输出：投影后的向量 z （如 128 维）
- **关键作用**：专门吸收和过滤掉那些对对比任务有用但对通用识别没用的冗余信息（颜色、位置、纹理等低级信息）。保护前面的编码器能学到更纯净、更本质的语义特征
- **训练后扔掉投影头**——只用编码器 f 的输出作为下游任务的表征

3. 对比损失函数 (NT-Xent Loss):

- 全称：Normalized Temperature-scaled Cross Entropy Loss
- 核心思想：最大化同一图像不同增强视图（正样本对）之间的相似度，同时最小化不同图像（负样本对）之间的相似度

SimCLR 的损失函数详解

对于 batch 中的一对正样本 (i, j) （来自同一张原图的两个增广版本），损失定义为：

$$l_{i,j} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(\text{sim}(z_i, z_k)/\tau)}$$

其中：

- z_i, z_j 是经过投影头后的特征向量（先做 L2 归一化）
- $\text{sim}(z_i, z_j) = z_i^T z_j / (\|z_i\| \|z_j\|) = \text{余弦相似度}$
- τ 是温度参数（temperature），控制分布的锐度
- 分母对 batch 内所有 $2N$ 个样本（除了 i 自己）求和

直观理解：把正样本对的相似度（分子）和所有负样本对的相似度（分母）对比。正样本的相似度越高、负样本的相似度越低 → 损失越小。最终总损失是 batch 内所有正样本对损失的对称和。

下游任务应用

经过 SimCLR 训练后，**扔掉投影头 g** ，只用基础编码器 f 的输出作为图像表征。这些表征可以直接用于 Linear Probing（冻结编码器，只训练线性分类器）或微调。SimCLR 最惊人的结果是：**仅用 1% 的 ImageNet 标签微调，就能超越全监督 AlexNet 的性能。**

八、Skip-Gram 变体的计算公式

Word2Vec 两种变体

Word2Vec（Mikolov et al., 2013）是经典静态词嵌入模型。两种变体：

- **Skip-Gram**：给定中心词 c ，预测上下文词 o 的概率—— $P(o|c)$
- **CBOW (Continuous Bag of Words)**：给定上下文词，预测中心词的概率—— $P(c|o_1, o_2, \dots)$

PPT 以 Skip-Gram 为例展开讲解。

Skip-Gram 的目标函数

优化目标：最大化语料库中所有中心词-上下文词对的对数概率。

给定长度为 T 的文本序列，中心词位置为 t ，上下文窗口大小为 m ：

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T \sum_{-m \leq j \leq m, j \neq 0} \log P(w_{t+j} | w_t; \theta)$$

其中：

- w_t 是位置 t 的中心词
- w_{t+j} 是位置 $t+j$ 的上下文词（窗口半径 m ，如 $m=5$ 则看前后各 5 个词）
- θ 是模型中所有单词的向量参数

损失函数 = $-J(\theta)$ （最小化负对数似然）。

Skip-Gram 中 $P(o|c)$ 的计算——Softmax 公式

对于每个单词 w ，用两个向量表示：

- **中心词向量 v_w** ：当 w 充当中心词时使用
- **上下文向量 u_w** ：当 w 充当上下文词时使用

两套向量分开增加了模型的优化灵活性。

给定中心词 c 和上下文词 o ，条件概率用 Softmax 计算：

$$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$

其中：

- v_c 是中心词 c 的中心词向量
- u_o 是上下文词 o 的上下文向量
- V 是词表（所有单词的集合）
- 分子：中心词向量和上下文词向量的内积（相似度）
- 分母：中心词向量和词表中**所有**单词的上下文向量内积之和

含义：在所有词中， o 与 c 的相似度（内积值）占比越大 $\rightarrow P(o|c)$ 越大 $\rightarrow$ 表示 c 出现后 o 出现的概率更高。

Skip-Gram 的优化——负采样（Negative Sampling）

问题：上述 Softmax 分母需要对词表 V 中所有单词计算内积。词表可能几十万甚至上百万——每一步都计算全部极其昂贵。

负采样解决方案：

- 不计算词表中所有词——只随机采样 K 个「负样本」（ K 通常 5~20）
- 将多分类问题转化为多个二分类问题
- 目标：真实上下文词 $\rightarrow$ 正样本（概率高），随机采样词 $\rightarrow$ 负样本（概率低）

负采样的目标函数：

$$J_{neg} = \log \sigma(u_o^T v_c) + \sum_{k=1}^K \mathbb{E}_{w_k \sim P_n(w)} [\log \sigma(-u_{w_k}^T v_c)]$$

其中：

- σ 是 sigmoid 函数
- $u_o^T v_c$ 是中心词 c 和真实上下文词 o 的内积（正样本，sigmoid 后应接近 1）
- $u_{w_k}^T v_c$ 是中心词 c 和随机采样的负样本词 w_k 的内积（负样本，取负号后 sigmoid 应接近 0）
- $P_n(w)$ 是负采样分布，正比于词频的 0.75 次方—— $P_n(w) \propto \text{count}(w)^{0.75}$

为什么用 0.75 次方：降低高频词（如「the」「a」）被采样的概率，提高中低频词被采样的概率。高频词被采样太多会主导训练，但它们本身无语义信息（停用词）。

第三板块：生成模型（第九章）

九、变分自编码器（VAE）的数学原理

VAE 的优化目标——最大似然估计

初始条件：已知一批观测数据 x （真实图像）。

目标：找到最优模型参数 θ ，使得已知观测数据在模型分布 P_θ 中出现的概率最大：

$$\theta^* = \arg \max_{\theta} \sum_x \log P_{\theta}(x)$$

$P_\theta(x)$ 的潜变量建模：

$$P_{\theta}(x) = \int P(z)P_{\theta}(x|z)dz$$

- $P(z)$ = 先验分布，一般取标准正态分布 $N(0, I)$
- $P_{\theta}(x|z)$ = Decoder，根据潜变量 z 生成图像 x 的概率——由神经网络参数 θ 控制

困难：上述积分需要对 z 在整个 128 维隐空间中积分——数值上不可行。

VAE 的变分推断——引入编码器

为解积分的难题，VAE 引入**编码器网络 $Q_\phi(z|x)$** 来近似真实的隐变量后验分布 $P(z|x)$ 。这就是「变分推断」——也是 VAE 中「变分 Variational」的由来。

总优化目标是让解码器分布 $P_{\theta}(x|z)$ 和隐变量真实分布 $P(z|x)$ 同时尽可能准，但后者不可解——用编码器分布 $Q_\phi(z|x)$ 近似替代（高斯分布）：

$$Q_\phi(z|x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi^2(x))$$

VAE 的最终损失函数——ELBO

将两个目标合并后推导得到最终的证据下界（ELBO）损失函数：

$$\mathcal{L}(\theta, \phi; x) = \mathbb{E}_{z \sim Q_\phi(z|x)} [\log P_{\theta}(x|z)] - D_{KL}(Q_\phi(z|x) \| P(z))$$

等价地，**最小化**以下损失：

$$\mathcal{L}_{VAE} = \mathcal{L}_{recon} + \mathcal{L}_{KL}$$

损失函数的两大部分

第一部分——重建损失 (Reconstruction Loss):

$$\mathcal{L}_{recon} = -\mathbb{E}_{z \sim Q_{\phi}(z|x)} [\log P_{\theta}(x|z)]$$

实际计算时从编码器分布 $Q_{\phi}(z|x)$ 中采样 z , 然后用 Decoder 重建 $\hat{x}$ 。如果 P_{θ} 取高斯分布, 重建损失等价于 MSE:

$$\mathcal{L}_{recon} = \|x - \hat{x}\|^2$$

约束: Decoder 能从采样的隐变量 z 中尽可能精确地还原出原始图像 x 。

第二部分——KL 散度正则项 (Kullback-Leibler Divergence Regularization):

$$\mathcal{L}_{KL} = D_{KL}(Q_{\phi}(z|x) \| P(z))$$

当先验 $P(z) = N(0, I)$ 且编码器 $Q_{\phi}(z|x) = N(\mu, \sigma^2)$ 时, KL 散度有解析解 (不需要采样/估计):

$$D_{KL}(\mathcal{N}(\mu, \sigma^2) \| \mathcal{N}(0, 1)) = \frac{1}{2} \sum_{d=1}^D (\mu_d^2 + \sigma_d^2 - \log \sigma_d^2 - 1)$$

- D 是隐空间的维度 (如 128)
- μ_d 是第 d 维的均值
- σ_d^2 是第 d 维的方差

约束: 编码器输出的分布 $Q_{\phi}(z|x)$ 不要偏离标准正态先验 $P(z)=N(0,I)$ 太远。

总损失: $\mathcal{L} = \mathcal{L}_{recon} + \lambda \cdot \mathcal{L}_{KL}$, λ 是平衡两项的权重系数 (博弈关键)。

十、正则项损失——KL 散度的关键作用

为什么 KL 散度不能被去掉

思想实验——如果只保留重建损失 MSE, 删掉 KL 散度:

神经网络为了最小化 MSE, 会通过让方差 $\sigma^2 \rightarrow 0$ 来消除采样引入的重建不确定性。方差被压到 0 $\rightarrow$ 所有图像的编码退化成一个确定性的点 ($\sigma=0$) $\rightarrow$ VAE 塌缩成传统自编码器。隐空间重新变成孤立、不连续的点分布——失去了所有生成新图像的能力。

KL 散度的作用机制:

1. 强制每一个 Encoder 输出的分布都向标准正态分布 $N(0, I)$ 靠近
2. 防止分布收缩为点 ($\sigma \rightarrow 0$), 维持分布的宽度
3. 让不同图像的编码分布之间有重叠——填补隐空间的「断层」
4. 方便生成时从 $N(0, I)$ 随机采样——绝大多数采样点都落在至少一个训练图像编码分布的覆盖范围内

VAE 训练的博弈

重建损失过强 $\rightarrow$ 隐空间离散、丧失生成多样性: Encoder 把每个图像编码成极窄分布, 不同图像的分布互不重叠, 隐空间破碎。

KL 散度过强 $\rightarrow$ 「后验崩溃」Posterior Collapse: Encoder 发现「对所有输入都输出 $N(0, I)$ 是最好的策略 (KL 散度为 0)」, 解码器收到的 z 和输入 x 毫无关系, 生成的图像是所有训练图像的平均——极其模糊。

VAE 训练的本质就是在寻找重建精度和生成多样性之间的最优平衡。 λ (KL 散度权重) 是核心工程超参。

十一、训练困境与重参数化技巧

训练困境——采样操作不可微

VAE 的诞生路线是：Encoder 输出 μ 和 $\sigma \rightarrow$ 从 $N(\mu, \sigma^2)$ 中随机采样得到 $z \rightarrow$ Decoder 用 z 重建 $\hat{x} \rightarrow$ 计算损失 = MSE + KL。

但问题是：从高斯分布 $N(\mu, \sigma^2)$ 中随机采样 z 是一个不可微操作。反向传播的梯度流在「采样」这一步彻底断裂：

MSE $\rightarrow$ Decoder $\rightarrow z \rightarrow$ 采样 (X 梯度断裂) $\rightarrow \mu, \sigma$

Encoder 的参数 μ 和 σ 收不到任何关于「应该怎么调整能让重建更好」的梯度信号。整个端到端训练无法进行。

解决方案——重参数化技巧 (Reparameterization Trick)

核心思想：将隐变量采样的随机性转移给不参与网络更新的外部随机噪声 ϵ 。

公式：

$$z = \mu + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, 1)$$

- ϵ 从外部随机采样 ($N(0,1)$)，不被梯度追踪，它只是一个「给定的观测值」
- μ 和 σ 是 Encoder 输出的确定性参数
- z 对 μ 和 σ 的梯度是： $\partial z / \partial \mu = 1, \partial z / \partial \sigma = \epsilon$

梯度链变为：

MSE $\rightarrow$ Decoder $\rightarrow z = \mu + \sigma \cdot \epsilon \rightarrow \mu, \sigma$ (梯度无障碍传递)

整个流程的随机性被局限在外部的 ϵ 中，而网络可学习的部分 (μ 和 σ) 通过线性变换与 z 连接，梯度能够畅通无阻地反向传播。

重参数化技巧的影响：这个简单的公式 $z = \mu + \sigma \cdot \epsilon$ 是 VAE 能从理论走向工程落地的最关键数学技巧。没有它，VAE 无法端到端训练。

十二、去噪模型的设计 (扩散模型)

背景

扩散模型的核心是学习「去噪」——每一步去掉含噪图像中的噪声以还原清晰图像。去噪模型学习的是什么？（预测目标是什么？）

思路一：神经网络直接预测加噪前的图像

- 输入：含噪图 x_t 、时间步 t
- 输出：去噪后的清晰图像 x_{t-1}
- 问题：图像分布极其复杂——猫、狗、人脸、风景.....类别多样、结构差异大。一个模型要学会在任意噪音级别上去预测任意类别的原始图像模样——学习难度极高！模型很难学到通用预测

思路二（主流——最终正确路径）：神经网络预测噪声而非图像

- 输入：含噪图 x_t 、时间步 t
- 输出：预测此步加入的噪声 $\epsilon_\theta(x_t, t)$
- **为什么更优**：无论原始图像是猫、狗还是人脸，加噪过程**始终是同一类高斯噪声**——独立高斯样本。噪声服从固定的、简单的分布。预测「哪部分是噪声」远比预测「这是什么物体」简单
- 模型不需要理解图像内容——只需要学会区分「哪里是噪点」，像去噪滤镜——不认人只识噪讯

去噪模型内部架构设计

条件信息 t （时间步）也被馈入模型（通常通过正弦嵌入+被注入每个残差模块），告诉模型当前处于加噪的哪个阶段——初期噪大、仅轮廓；后期噪极小、细节纹理显现时分清微妙变化。这使同一模型能够在不同噪音等级下运行——通用性极高。

十三、扩散模型的训练流程

核心训练公式——单步加噪

利用独立高斯噪声的可加性，从原图 x_0 直接一步合成任意第 t 步的含噪图 x_t ：

$$x_t = \sqrt{\alpha_t} \cdot x_0 + \sqrt{1 - \alpha_t} \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

- α_t 是噪声强度参数（随 t 增加从 1 到 0）——控制原图信息和噪声信息之间的比例
- ϵ 是标准噪声（一维独立高斯样本）
- $\sqrt{\alpha_t}$ ：原图信号保留比例
- $\sqrt{1 - \alpha_t}$ ：噪声信号混入比例

单步加噪的推导：多步独立高斯噪声逐步叠加 = 等价于一个总体合并噪声的等效高斯——因此任意时间步的含噪图像可一步直接计算，无需在前训练中逐步慢加噪 1000 次。

训练的完整流程

1. **随机采样**：从数据集中随机取一张真实图像 x_0 ；随机取一个时间步 $t \sim \text{Uniform}(1, T)$ ；随机生成一个高斯噪声 $\epsilon \sim \mathcal{N}(0, I)$
2. **一步合成含噪图**：用上面公式计算 $x_t = \sqrt{\alpha_t} \cdot x_0 + \sqrt{1 - \alpha_t} \cdot \epsilon$
3. **噪声预测**：将 x_t 和 t 送入噪声预测网络 ϵ_θ ，让网络预测所加的真实噪声 ϵ
4. **计算损失**： $L = || \epsilon_\theta(x_t, t) - \epsilon ||^2$ ——噪声预测值与真值的 MSE
5. **更新参数**：反向传播，更新噪声预测网络参数 θ

训练策略——随机采样时间步

为什么不能按 1→2→3→...→T 的顺序一步步训练：

- 如果按顺序训练——前期只学低噪条件（初始几步）、后期只学高噪条件（末尾几步）
- 模型出现**灾难性遗忘**——学会了后面的忘记前面的
- 深度学习训练需要样本随机打乱——确保每次梯度下降信号覆盖全局

解决方案——随机采样时间步：每个训练迭代随机抽取一个 t ，马上用单步公式生成 x_t 送进模型训练。这样模型在每个训练 batch 都能同时学到从轻噪到重噪的各种噪音条件下的去噪行为——全局稳定、无遗忘。

关于随机打乱必要性的公式推导

多步加噪原本需要 t 步，训练代价过高。利用高斯可加性推导如下递推关系：

- 1. $x_1 = \sqrt{\alpha_1}x_0 + \sqrt{1 - \alpha_1}\epsilon_1$
- 2. $x_2 = \sqrt{\alpha_2}x_1 + \sqrt{1 - \alpha_2}\epsilon_2 = \sqrt{\alpha_1\alpha_2}x_0 +$ 两个独立高斯的线性组合
- 3. 由高斯独立性, $\sqrt{\alpha_2(1 - \alpha_1)} + \sqrt{1 - \alpha_2} \rightarrow$ 合并为一个等效高斯噪声 $\sqrt{1 - \alpha_1\alpha_2} \cdot \epsilon$
- 4. 递推至 t 步—— $x_t = \sqrt{\alpha_t} \cdot x_0 + \sqrt{1 - \alpha_t} \cdot \epsilon$ (证毕)

最终训练总损失 = $||\epsilon_\theta(x_t, t) - \epsilon||^2$ (预测噪声与真实噪声的 L2 损失)。

十四、扩散模型的推理（生成）流程

推理流程

第一步——给定时间 t 及该时刻含噪图 x_t ，神经网络预测噪声并减去：

- 输入：时间步 t 、该时刻的含噪图 x_t （或隐变量状态）
- 输出：模型预测的当前噪声 $\hat{\epsilon} = \epsilon_\theta(x_t, t)$
- 利用预测噪声去除该时刻混入的噪音分量 $\rightarrow$ 得到（近似）原始的 $x_{\{t-1\}}$

第二步——给去噪后结果加入少量随机噪声（Stochastic / Random Noise Injection）：

- 生成 $x_{\{t-1\}}$ 的公式包含从 $N(0, I)$ 采样一小份随机噪声，加到降噪输出上
- 作用：1) 增加生成多样性——每次生成结果略有不同避免结果唯一；2) 打破局部预测误差累积链——加入微量扰动防止错误顺延到后续步。

完整 T 步迭代

从纯噪声 $x_T \sim N(0, I)$ 开始：

for $t = T \rightarrow 1$ ：

 预测噪声 $\hat{\epsilon}_t = \epsilon_\theta(x_t, t)$

 利用 $\hat{\epsilon}_t$ 和随机噪声计算 $x_{\{t-1\}}$

最终 x_0 = 清晰图像（生成成功）

T （去噪步数）的选择（如 50~1000）代表了细度 vs 速度的 trade-off——多步 $\rightarrow$ 更细更准但慢；少步 $\rightarrow$ 更快但可能留噪影。

课后速查表

VAE 核心公式

公式	含义
$z = \mu + \sigma \cdot \epsilon$	重参数化：将随机性转移给外部噪声 ϵ

公式	含义
$L = \text{MSE}(x, \hat{x}) + \text{KL}(\mathcal{N}(\mu, \sigma^2) \parallel \mathcal{N}(0, I))$	VAE 总损失 = 重建损失 + KL 正则
$\text{KL} = \frac{1}{2} \sum (\mu^2 + \sigma^2 - \log \sigma^2 - 1)$	KL 散度解析解
$P(x) = \int P(z) P(x z) dz$	边际似然（潜变量积分，不可直接求）

扩散模型核心公式

公式	含义
$x_t = \sqrt{\alpha_t} \cdot x_0 + \sqrt{1 - \alpha_t} \cdot \epsilon$	单步加噪合成含噪图
$L = \ \epsilon_\theta(x_t, t) - \epsilon\ ^2$	噪声预测损失
$z = \mu + \sigma \cdot \epsilon$	重参数化（同 VAE）

Skip-Gram 核心公式

公式	含义
$P(o c) = \exp(u_o^T v_c) / \sum_w \exp(u_w^T v_c)$	Softmax 求上下文概率
$J_{neg} = \log \sigma(u_o^T v_c) + \sum_k \log \sigma(-u_k^T v_c)$	负采样目标函数

SimCLR 损失

公式	含义
$l_{i,j} = -\log(\exp(\text{sim}(z_i, z_j)/\tau) / \sum_k \exp(\text{sim}(z_i, z_k)/\tau))$	NT-Xent 对比损失

RAG 架构对比

架构	检索器更新	LLM更新	代表方法
黑盒-无微调	X	X	In-Context RALM
黑盒-检索器微调	✓	X	REPLUG
白盒-仅微调LLM	X	✓	RETRO
白盒-协同微调	✓	✓	ATLAS